

Computable longitudinal patient journeys from structured and unstructured EHR data

Technical documentation of the two released code repositories (Kim, Foty et al., Nature Medicine 2026, doi:10.1038/s41591-026-04695-x)

Compiled 2026-10-08

How to use this document

One-page summary

Part I — Extraction validation repository (`glp1_nature2026_extraction_validation`)

Architecture — Part I (extraction validation)

Code reference — Part I (extraction validation)

Data schema — Part I (extraction validation)

Paper coverage and modularity — Part I (extraction validation)

Reuse guide — running this validation on a different clinical question or on MIMIC

Part II — GLP-1 outcome analysis repository (`glp1_nature2026_a1c_analysis`)

Architecture — Part II (GLP-1 outcome analysis)

Code reference — Part II (GLP-1 outcome analysis)

Data schema — Part II (GLP-1 outcome analysis)

Paper coverage and modularity — Part II (GLP-1 outcome analysis)

Reuse guide — same pipeline, different clinical question, or MIMIC

How to use this document

This is a single-file compilation of the `docs/` folders of the two public repositories that accompany the paper:

Repository	Licence	Covers
Zacharyr41/glp1_nature2026_extraction_validation (Part I)	Apache 2.0	Validation of the LLM extraction platform: physician review, inter-rater agreement, BiolarkGSC+ benchmark, phenotype-prevalence figure
Zacharyr41/glp1_nature2026_a1c_analysis (Part II)	MIT	The GLP-1 outcome pipeline: cohort construction, persistence censoring, GEE spline trajectories, Kaplan-Meier / Cox, note-derived assessments

It is written to let another project **synthesise** the design rather than re-read the code. If you are building a comparable pipeline, read in this order:

1. **Part II > Architecture §3 (“The one statistical pattern”)** — the entire analysis is one GEE recipe applied in variants.
2. **Part II > Data schema §1** — the single wide CSV is the contract between the proprietary platform and the released code. Everything upstream of it is unavailable; everything downstream is released.
3. **Part II > Reuse guide §4** — the ordered list of components that must be written from scratch on new data (cohort builder, baseline-window selector, plausibility cleaning, exposure evidence rows, note extraction).
4. **Part I > Architecture §1** — where the released validation sits relative to the unreleased extraction platform, and **Part I > Reuse guide §3** for what a MIMIC-based extraction layer needs.
5. **Paper coverage** in both parts — which published tables and figures reproduce, and which cannot.

One-page summary

The paper’s system. De-identified ambulatory EHR (35 M patients) → agentic LLM extraction of atomic statements from notes (medications, phenotypes, symptoms, assessments, adherence, family history, hospitalisations), each with a 0–3 verification score and source span → ontology grounding (UMLS, HPO, RxNorm, ICD, LOINC, SNOMED) → patient-centred knowledge graph → agentic analytic layer that emits study-specific wide CSVs → statistical analysis. Only the last stage and the evaluation of the extraction stage are released.

What reproduces without data. Table 1 (model-comparison block), Extended Data Table 1 (inter-rater κ) and Extended Data Table 5 (BiolarkGSC+ benchmark), all from Part I’s frozen labels and predictions. Every number involving patients requires the licensed CSV.

What is not released anywhere. Extraction prompts (fragments only), delta tracking, the verification judge, ontology mapping, concept discovery (output JSON only), the knowledge graph, fine-tuning, the physician review interface, all data cleaning and cohort-entry logic described in the Methods, and the deliverable formatters.

Modularity. Rules are modular (persistence, spline df, reference level, estimability, confidence level each live in one tested module). Scripts are not: the GEE recipe is re-implemented in roughly fourteen near-duplicate files with hard-coded labels, thresholds and paths, and there is no configuration layer.

Known defects (verified against the committed code; none affect primary estimates): an unset `$FORCE` aborts the structured-only runner at step8b; the three-way stratification scripts ignore `--spline-df`; Cox covariates for the weight outcome come from the HbA1c cohort file; the runner’s path override for the frequency scripts is ineffective.

Part I — Extraction validation repository (glp1_nature2026_extraction_validation)

Architecture — Part I (extraction validation)

This repository is **Part I** of the code behind *Computable longitudinal patient journeys from structured and unstructured EHR data* (Kim, Foty et al., *Nature Medicine* 2026, doi:10.1038/s41591-026-04695-x). It contains the **evaluation** of the authors' LLM extraction platform, not the platform itself. Part II (the GLP-1 outcome analysis) is the sibling repository `glp1_nature2026_a1c_analysis`.

1. Where this repo sits in the paper's system

The paper describes a five-layer platform. Only the shaded layer is released here; the rest is proprietary (see the Code availability statement) or lives in Part II.

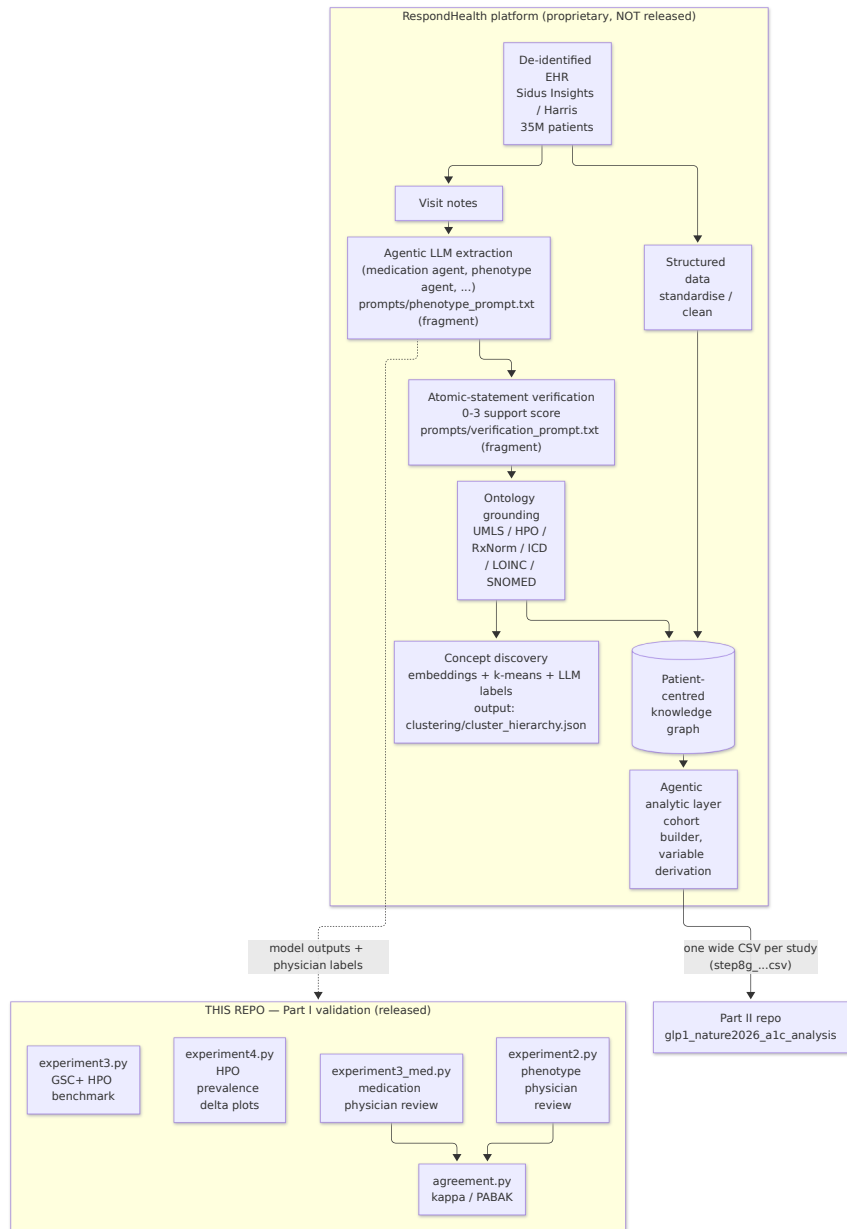


Diagram 1 — ARCHITECTURE.md

2. Inside this repo

Everything is batch: `code/run.sh` runs the unit tests and four experiment scripts, each reading frozen inputs under `data/` and writing a text log or PNG under `results/`. There is no shared Python package other than `code/agreement.py`; each experiment is a standalone script with hard-coded relative paths (`os.path.join(script_dir, '..', 'data')`).

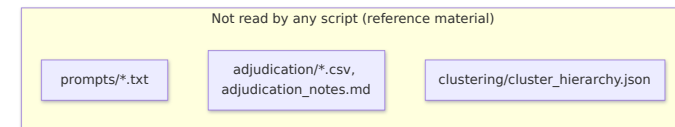
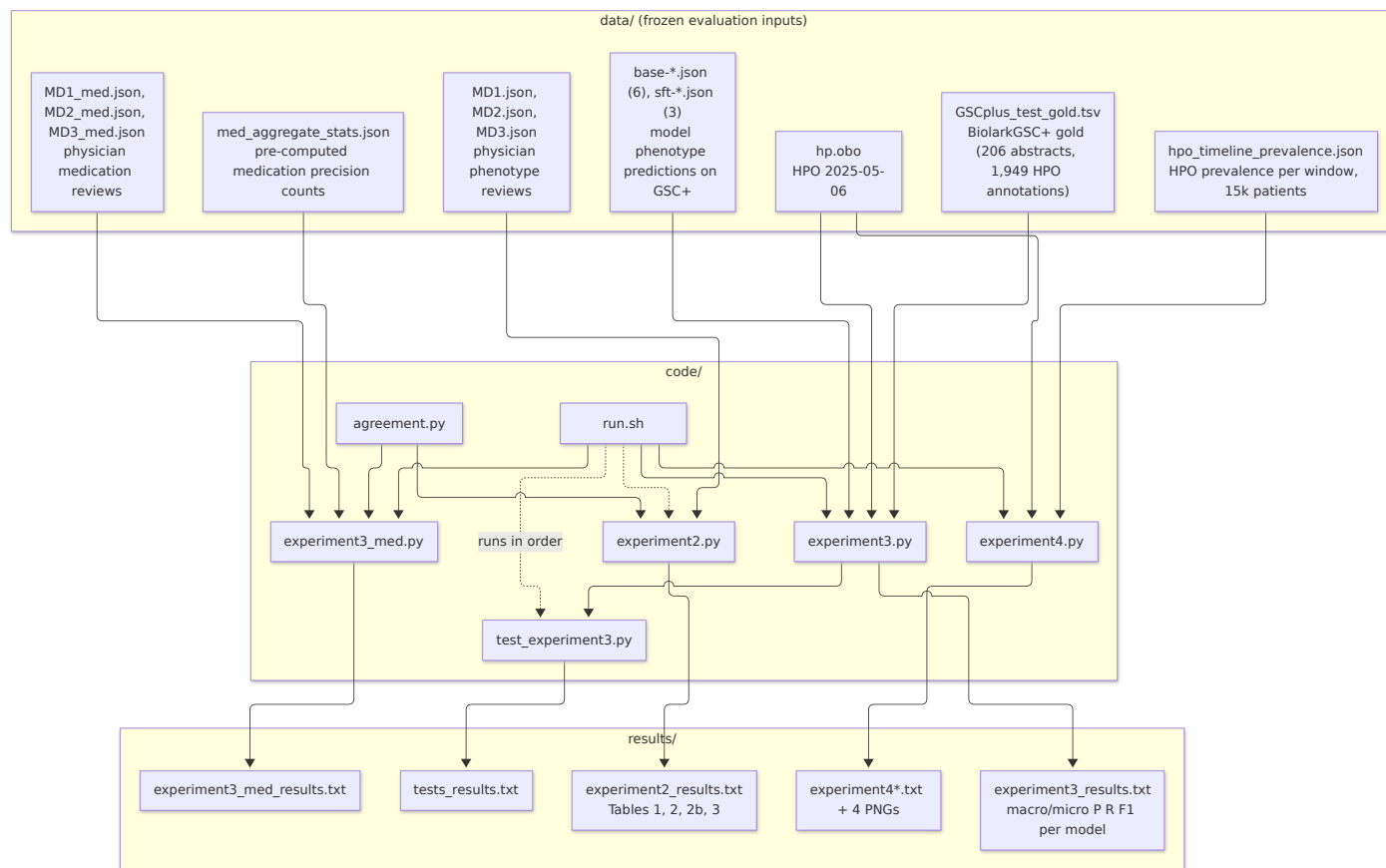


Diagram 2 — ARCHITECTURE.md

Execution order (code/run.sh)

Step	Command	Output	Paper item
1	python3 -m unittest discover -s . -p 'test_*.py'	results/tests_results.txt	—
2	python3 experiment2.py	results/experiment2_results.txt	Table 1 (phenotype rows), Extended Data Table 1 (phenotype κ)
3	python3 experiment3_med.py	results/experiment3_med_results.txt	Table 1 (medication rows), Extended Data Table 1 (medication κ)
4	python3 experiment3.py	results/experiment3_results.txt	Extended Data Table 5 (BiolarkGSC+)
5	python3 experiment4.py --json ... --data_dir ../data --output_dir ../results	results/experiment4_results.txt, two PNGs	Exploratory phenotype-prevalence figure (not a numbered display item)
6	same with --no-normalize --suffix experiment4_unnormalized	sensitivity figures	—

run.sh uses set -euo pipefail, records each step’s exit code, echoes the last 20 lines of a failing log to stderr, keeps running the remaining steps, and exits non-zero at the end if any step failed.

3. Design properties worth knowing

- **Reproducible offline.** No network, no model calls, no PHI. All LLM outputs were generated upstream and are frozen in data/. Running run.sh reproduces every number in results/.
- **Scoring is independent of ontology assignment.** In experiment3.py the gold HPO ID drives synonym and hierarchy expansion; the model’s own hpo_matched_name is only an extra surface string. Extraction quality and normalisation quality are therefore separately measurable.
- **One agreement estimator.** agreement.py is the only kappa implementation. It reports Cohen’s κ and PABAK side by side with p_o and p_e and flags degenerate pairs, so the medication table’s $\kappa = 0.000$ at 95.8 % agreement is explained rather than hidden.
- **Minimal dependencies.** Only experiment4.py needs third-party packages (numpy, matplotlib). The other three experiments and the tests are standard-library Python.
- **No configuration layer.** Paths, model lists, reviewer file names, source keys, and display names are literals inside each script. See REUSE_GUIDE.md for the exact edit points.

See CODE_REFERENCE.md for function-level documentation, DATA_SCHEMA.md for every input and output format, PAPER_COVERAGE.md for what the paper reports that is and is not reproducible here, and REUSE_GUIDE.md for how to run the same validation on another clinical question or on MIMIC.

Code reference — Part I (extraction validation)

Function-level documentation for everything under `code/`. Line numbers refer to the committed files. Paper-table attributions are inferred from content; the only in-repo cross-reference is `code/CHANGES.md`, which ties `experiment3.py` to “Table S1a” (Extended Data Table 5 in the published paper).

code/agreement.py — shared inter-rater agreement estimator (99 lines)

Imported by `experiment2.py` and `experiment3_med.py` so both agreement tables use one estimator.

Name	Line	Role
<code>N_CLASSES = 3</code>	24	Label encoding: 0 = Present, 1 = Absent, 2 = Rejected
<code>TOL = 1e-10</code>	25	Numerical tolerance for “p_e is 1”
<code>confusion_matrix(items, r1, r2)</code>	28	3×3 matrix over items both reviewers scored; returns (matrix, n)
<code>agreement_stats(matrix, n)</code>	37	p_o (diagonal / n), p_e ($\sum \text{row}_i \cdot \text{col}_i / n^2$), Cohen’s κ ($(p_o - p_e) / (1 - p_e)$) or None when p_e saturates, PABAK $2 \cdot p_o - 1$, and degenerate = constant marginal or p_e saturated
<code>pairwise_agreement(items)</code>	68	Runs <code>agreement_stats</code> for every reviewer pair discovered in <code>items</code> ({item_key: {reviewer: label}})
<code>format_kappa(stats)</code>	87	"undefined" when κ is None
<code>HEADER, format_row(stats)</code>	91, 95	Fixed-width table rendering: Rater Pair, N, Agreement (%), p0, pe, Cohen's K, PABAK, Degenerate

Why PABAK: if one rater assigns every item to the same class, $p_o = p_e$ exactly and $\kappa = 0$ regardless of observed agreement; if both do, κ is 0/0. The module tests that condition directly rather than comparing the magnitudes of κ and PABAK (the pre-review heuristic).

code/experiment2.py — physician review of phenotype extraction (271 lines)

Purpose. Inter-rater agreement among three physicians (MD1-MD3) and precision / negation-error metrics for two phenotype extractors on 24–27 real clinical visits: `PhenoTagger` (non-LLM baseline) and `Verified` (the authors’ GPT pipeline, displayed as “Ours (GPT)”). Feeds **Table 1** (phenotype rows of the comparison block) and **Extended Data Table 1** (phenotype κ).

CLI. None. Reads `../data/MD{1,2,3}.json` relative to the script (lines 135–142); exits 1 if any is missing.

Inputs. Each MD file is a list of review records; fields consumed: `reviewer`, `source`, `is_missing`, `is_correct`, `original_negation`, `corrected_negation` (metrics), and `visit_key`, `phenotype_name`, `start_pos`, `end_pos`, `label` (agreement). See `DATA_SCHEMA.md`.

Outputs (stdout; `run.sh` redirects to `results/experiment2_results.txt`):

Table	Columns
Table 1: Inter-Rater Agreement	Phenotypes, Rater Pair, N, Agreement (%), p0, pe, Cohen's K, PABAK, Degenerate per source × pair
Table 2: Phenotype Extraction Performance (aggregated)	Model, N (extracted), N (missing), Precision (%), Negation Error (%), Other Error (%)
Table 2b: Emitted negation distribution	Model, N (extracted), Asserted present, Asserted negated
Table 3: Per-Reviewer Performance Detail	per source: Strict P/R/F1, Entity P/R/F1, Negation Error Rate, one column per reviewer

Metric definitions (`calculate_performance_metrics`, lines 14–70), per reviewer × source:

```
is_missing      -> missing_count (FN: phenotype in note, not extracted)
else total_extracted += 1
  is_correct    -> correct_count
  elif corrected_negation != original_negation -> negation_errors
  else          -> other_errors
tp_strict = correct_count; tp_entity = tp_strict + negation_errors
P_strict = tp_strict / total_extracted
R_strict = tp_strict / (tp_strict + missing + negation_errors)
P_entity = tp_entity / total_extracted
R_entity = tp_entity / (tp_entity + missing)
negation_error_rate = negation_errors / tp_entity
```

Points to note: “strict” treats a negation error as both a wrong extraction and a missed entity; “entity” ignores polarity; `other_errors` (wrong concept) never enter a recall denominator; recall is only against reviewer-flagged `is_missing` rows. `calculate_aggregate_performance` (lines 73–120) pools across reviewers, so one extraction reviewed by three physicians counts three times.

Agreement. `build_items` (lines 123–131) drops `is_missing` rows and keys each item as `"{visit_key}|{phenotype_name}|{start_pos}|{end_pos}"` → `{reviewer: label}`.

Hard-coded. `SOURCE_DISPLAY = {'PhenoTagger': 'PhenoTagger', 'Verified': 'Ours (GPT)'}` (lines 8–11); file names MD1-MD3; the three-class label encoding.

Functions. `calculate_performance_metrics(reviews, reviewer_name, source_name)` :14 · `calculate_aggregate_performance(reviews, source_name)` :73 · `build_items(source_reviews)` :123 · `get_agreement_stats()` :134 (main; loads, prints all tables, returns exit code).

code/experiment3_med.py — physician review of medication extraction (125 lines)

Purpose. Same design as experiment2 for medications: agreement on three extractors — `med_ner` (“Bert Large Med NER”), `disease_ner_meds` (“BioNER”), `medications` (“Ours (GPT)”) — plus an aggregate precision table. Feeds the medication rows of **Table 1** and **Extended Data Table 1**.

Inputs. `data/MD{1,2,3}_med.json` (fields `source`, `reviewer`, `is_missing`, `visit_key`, `finding_name`, `finding_index`, `label`) and `data/med_aggregate_stats.json` (`{section: {total_extracted, correct, neg_err, other_err}}`). The aggregate counts come from a wider reviewer set than the three MD files (docstring lines 16–18) and cannot be recomputed from them. No recall is reported because the source carries no missing count.

Algorithm. Agreement identical to experiment2 via `agreement.py`, item key `"{visit_key}|{finding_name}|{finding_index}"` (no character offsets). Aggregate: `precision = correct/total_extracted`, `negation_error_rate = neg_err/(correct+neg_err)`, `other_error_rate = other_err/total_extracted`.

Headline (results file). “Ours (GPT)” pairs are degenerate: MD1–MD2 and MD1–MD3 are 22/22 Present (κ undefined, PABAK 1.000); MD2–MD3 has a constant MD2 marginal (κ 0.000 at 95.8 % agreement, PABAK 0.917). Baselines are non-degenerate (κ 0.78–1.00).

Functions. `calculate_aggregate_performance_from_json(data_dir)` :15 · `build_items` :48 · `main()` :59.

code/experiment3.py — BiolarkGSC+ HPO benchmark (452 lines)

Purpose. Scores nine LLM phenotype-prediction files (six base, three supervised-fine-tuned) and three two-model unions against the GSC+ test gold standard (206 PubMed abstracts, 1,949 HPO annotations), raw and after a post-processing validity filter. Feeds **Extended Data Table 5** (the “LLMs (base)”, “Distilled SFT”, “Post Proc (PP)” and “Multi-Agent” blocks; the “Encoders & early LLMs” rows are quoted from the literature).

CLI. None. `hp.obo` is parsed at import time (lines 82–83), so importing the module (as the tests do) loads the ontology.

Inputs.

- `data/GSCplus_test_gold.tsv` — PubTator-style blocks: PMID line, abstract line, then `start\tend\tsurface\thpo_id` lines, blank-line terminated. `parse_gold_standard` (lines 197–253) keeps (`surface`, `hpo_id`) and validates but discards offsets. Seven documents have zero annotations and are retained as empty.
- `data/hp.obo` — `load_hpo` (lines 44–75) reads [Term] stanzas: `id`, `name`, `alt_id`, `synonym` (quoted string), `is_a`, `is_obsolete`. 19,650 terms.
- `data/{base,sft}*.json` — `{pmid: [item, ...]}`; only `item["name"]` and `item["hpo_matched_name"]` are used. `negation` and `status` are present but **not scored**.

Matching (`check_match`, lines 154–190). Prediction strings are normalised (lower-case, punctuation → space) and tokenised to frozensets minus `FUNCTION_WORDS` (line 14). A prediction matches a gold annotation if any layer fires:

- exact** — a prediction token-set equals the gold surface form, the canonical HPO label, or any `hp.obo` synonym of the gold ID (`gold_expansion`, lines 130–136).
- subset** — strict token-subset in either direction, where the smaller side carries at least one token outside `GENERIC_TOKENS` (lines 19–23: `abnormality`, `disorder`, `disease`, `syndrome`, `finding`, `increased`, `decreased`, ... and plurals).
- hierarchy** — the prediction string resolves through `HPO_LABEL_TO_IDS` to the gold concept or a *descendant* of it (`hpo_ancestors`, cached transitive `is_a` closure, lines 93–106). Predicted ancestors of the gold term do not match unless layer 2 fires.

`difflib` similarity was removed after review because it scored microcephaly/macrocephaly 0.917 and deafness/hearing loss 0.50 (`CHANGES.md` §2).

Per-class accounting (`update_name_class_stats`, lines 256–279). Within a document, each gold annotation is a class keyed by its normalised surface string: TP if any prediction matches, else FN. Each prediction that matches nothing in the document is an FP under its own normalised name. Many-to-one matches are allowed in both directions.

Aggregation (`print_macro_row`, lines 363–389). Macro precision = mean of `tp/(tp+fp)` over classes with ≥ 1 prediction; macro recall = mean of `tp/(tp+fn)` over classes with ≥ 1 gold annotation; **macro F1 = harmonic mean of those two means** (not the mean of per-class F1); micro F1 = `2TP/(2TP+FP+FN)` over pooled counts. Printed `TP/FP/FN` are the micro sums.

Post-processing filter (`is_valid_phenotype`, lines 297–314). Drops a prediction whose normalised name matches `\bnormal\b`, `\bunremarkable\b`, contains no `abnormal`, matches any `GENETIC_KEYWORDS` (`mutation`, `deletion`, `duplication`, `translocation`, `chromosome`, `gene`, `allelic`, `mosaicism`, `trisomy`, `monosomy`, `karyotype`, with optional plural, word-bounded), or matches `\bsyndromes?\b`. These encode the GSC+ protocol (phenotypic-abnormality branch only).

Multi-model union (lines 408–447). `combined_models` pairs `base-claude-4.5` + `base-gpt-5.2-thinking-low`, `base-gpt-4.1-mini` + `base-qwen-4b-instruct`, `sft-gpt-4.1-mini` + `sft-qwen-4b-instruct`. Per document the two item lists are unioned and de-duplicated on (`name`, `hpo_matched_name`); no voting. This is the recall-maximising “multi-agent aggregation” of the paper.

Output. One table `Model | Type | Prec | Rec | F1 | microF1 | TP | FP | FN`, two rows per model (Base = raw, PP = post-processed).

Functions. `normalize` :26 · `tokenize` :30 · `load_hpo` :44 · `resolve_hpo_id` :86 · `hpo_ancestors` :93 · `gold_expansion` :130 · `prediction_tokens` :139 · `is_informative` :150 · `check_match` :154 · `parse_gold_standard` :197 · `update_name_class_stats` :256 · `is_valid_phenotype` :297 · `main` :321 (inner `process_and_report` :345, `print_macro_row` :363). Module globals `HPO_NAMES`, `HPO_SYNONYMS`, `HPO_PARENTS`, `HPO_ALT_IDS`, `HPO_OBSOLETE` :82, `HPO_ID_TOKENS`, `HPO_LABEL_TO_IDS` :110–111.

code/experiment4.py — HPO prevalence delta around GLP-1 initiation (439 lines)

Purpose. From a pre-aggregated per-window prevalence table, computes for every HPO code the cumulative after-minus-before prevalence delta at each day threshold, fits a slope, and plots the ten most negative and ten most positive. Reproduces `comparisonhpo2vis.py` from the proprietary pipeline (docstring). Exploratory figure; not a numbered display item in the final paper.

CLI (lines 348-388): `--json` (default `../data/hpo_timeline_prevalence.json`), `--top 10`, `--suffix experiment4`, `--output_dir .`, `--data_dir .` (where `hp.obo` is looked up; lines 398-399 override the module defaults unconditionally, so omit `--data_dir` and the legend shows raw HPO codes), `--no-normalize`.

Inputs. `hpo_timeline_prevalence.json` keys `meta.window_labels_before/after`, `meta.total_patients`, `meta.overall_event_counts`, `overall_cumulative_by_window[label].percent`. `overall_by_window` and `.counts` are unused. `hp.obo` for `id/name` only; `phenotype.hpoa` is a fallback that is not shipped.

Algorithm. - Thresholds = sorted distinct `|day|` from labels `prev_N / post_N` → `[30, 60, 90, 120, 180, 240, 300, 365]`. - Normalisation (`get_normalization_factors`): `factor[label] = total_patients / overall_event_counts[label]`, i.e. prevalence per unit of encounter volume; post-windows have ~3× more events than pre-windows, so normalisation damps the post side. `--no-normalize` is the sensitivity run. - Delta series: `y[t] = pct(post_t) - pct(prev_t)`, missing codes = 0. - Ranking: `np.polyfit(thresholds, y, 1)[0]`; top/bottom `--top` by slope. - `EXCLUDE_CODES = {"HP:0000819"}` (Diabetes mellitus) — excluded because it is the indication, not an outcome, and would swamp the delta scale (comment lines 43-47).

Outputs. `highlight_delta_cumulative_{down,up}_{suffix}.png` at 150 dpi; a log on stdout.

Functions. `is_excluded :62 · log :66 · load_results :70 · build_hpo_name_map_from_obo :79 · build_hpo_name_map_from_hpoa :112 · build_hpo_name_map :142 · extract_window_order :154 · label_to_day :161 · get_normalization_factors :170 · get_delta_cumulative_series :196 · make_highlight_delta_cumulative_lines :236 · main :335`.

code/test_experiment3.py — regression tests (148 lines)

Run: `python3 -m unittest discover -s code -p 'test_*.py' -v` (10 tests, all pass).

Class	Pins
TestGoldParser	206 documents, 1,949 annotations, first annotation of PMID 1003450 is “brachydactyly”, every annotation has an HPO ID, exactly 7 empty documents
TestMatcherRejectsOpposites	14 clinically opposite / wrong-concept pairs must not match (microcephaly vs macrocephaly, hypotonia vs hypertonia, ...)
TestMatcherAcceptsVariants	8 naming variants must match (seizures ↔ epilepsy, hearing impairment ↔ deafness, intellectual disability ↔ mental retardation, ...); bare “abnormality” must not match “abnormality of the eye”
TestValidityFilter	Keeps Abnormal EEG, renal agenesis, generalized hypotonia; drops normal findings, FBNI gene mutation, Marfan syndrome, trisomy 21

code/run.sh

Bash orchestrator described in `ARCHITECTURE.md`. `run_step LABEL OUTFILE CMD...` redirects stdout and stderr to `OUTFILE`, records the exit code, and prints the log tail on failure. Exit code of the script is non-zero if any step failed.

code/CHANGES.md

Ten numbered corrections made in response to code review, each with the defect, the fix, and the effect on reported numbers (for example: the gold parser dropped the first annotation of every document, 199/1,750 → 206/1,949; the `'gene'` in name substring filter was discarding `agenesis`). Read it before trusting any number in `results/` that predates the fix.

Data schema — Part I (extraction validation)

Every file under `data/`, `adjudication/`, `clustering/` and `prompts/`, with the fields each script consumes. Counts were verified by loading the committed files.

1. Physician phenotype reviews — `data/MD1.json`, `MD2.json`, `MD3.json`

JSON list, one record per model-extracted phenotype statement reviewed by one physician (MD1 n=1,336; MD2 n=1,374; MD3 n=1,025; 24 shared visits, MD3 reviewed 27).

Field	Type	Meaning	Used by
visit_key	str, e.g. VISIT-01	De-identified visit identifier	item key (experiment2)
source	str ∈ {PhenoTagger, Verified}	Which extractor produced the statement; <i>Verified</i> = the authors' GPT pipeline	experiment2
phenotype_name	str	Extracted surface string	item key
start_pos, end_pos	int	Character offsets in the (unreleased) note text	item key
reviewer	str ∈ {MD1,MD2,MD3}	One value per file	experiment2
original_negation	bool	Polarity asserted by the extractor (True = negated)	negation accounting
corrected_negation	bool	Polarity according to the physician	negation accounting
is_correct	bool	Physician verdict on the extraction	precision
is_missing	bool	True for phenotypes the physician found in the note that the extractor missed (reviewer-added rows)	recall; excluded from agreement
label	int ∈ {0,1,2} or null	Agreement class: 0 Present, 1 Absent (negated), 2 Rejected; null when <i>is_missing</i>	agreement.py

Observed joint structure: `label 0` ⇔ `is_correct` and not `corrected_negation`; `label 1` ⇔ `corrected_negation` (either a correct negated extraction or a negation error); `label 2` ⇔ not `is_correct` with unchanged polarity; `label null` ⇔ `is_missing`.

2. Physician medication reviews — `data/MD1_med.json`, `MD2_med.json`, `MD3_med.json`

JSON list (MD1 n=319; MD2 n=149; MD3 n=217).

Field	Type	Meaning	Used by
visit_key	str	as above	item key (experiment3_med)
source	str ∈ {med_ner, disease_ner_meds, medications}	BERT Large Med NER / BioNER / authors' GPT pipeline	experiment3_med
finding_name	str, e.g. Amlodipine 5 MG / valsartan 160 MG Oral Tablet [Exforge]	Extracted medication string (RxNorm-style for the GPT pipeline)	item key
finding_index	int	0-based position in the visit's extraction list	item key
reviewer, original_negation, is_correct, corrected_negation, is_missing, label	as in §1		only <code>label</code> , <code>is_missing</code> , <code>reviewer</code> , <code>source</code> are used

3. `data/med_aggregate_stats.json`

Pre-computed counts over a wider reviewer set than the three MD files; not recomputable here.

```
{ "med_ner": { "total_extracted": 344, "correct": 225, "neg_err": 13, "other_err": 106 },
  "disease_ner_meds": { "total_extracted": 179, "correct": 146, "neg_err": 6, "other_err": 27 },
  "medications": { "total_extracted": 221, "correct": 211, "neg_err": 5, "other_err": 5 }}
```

These are the `N` and precision values in the medication rows of Table 1.

4. `data/GSCplus_test_gold.tsv` — BiolarkGSC+ gold standard (third-party, see NOTICE)

PubTator-style text blocks separated by blank lines:

```
1003450                                <- PubMed ID (digits only)
A syndrome of brachydactyly (...)      <- abstract text
14 27 brachydactyly HP:0001156         <- start \t end \t surface \t HPO ID
...
```

206 documents, 1,949 annotations, 405 distinct HPO IDs. Seven documents carry no annotations within the phenotypic-abnormality branch and are retained as empty. `experiment3.py` uses columns 3 and 4 only.

5. data/hp.obo — Human Phenotype Ontology, release 2025-05-06 (third-party)

OBO 1.2, 19,650 [Term] stanzas. Fields parsed: id, name, alt_id, synonym, is_a, is_obsolete (experiment3); id, name (experiment4).

6. Model predictions on GSC+ — data/base-*.json (6 files), data/sft-*.json (3 files)

{pmid: [item, ...]} over 199 PMIDs (the 7 empty gold documents are absent). One item per extracted phenotype:

Field	Type	Meaning	Scored?
name	str	Extracted phenotype string	yes
hpo_matched_name	str	The pipeline’s downstream ontology match (HPO label)	used only as an extra surface string
negation	bool	Polarity	no
status	str (unknown, mild, moderate, severe, ...)	Severity	no
is_new_information	bool	Delta-extraction flag	no
details	str	Supporting sentence from the abstract	no
atomic_statement	str	Single-proposition restatement	no

Files: base-claude-4.5, base-gpt-4.1-mini, base-gpt-4.1-nano, base-gpt-5-mini-thinking-low, base-gpt-5.2-thinking-low, base-qwen-4b-instruct, sft-gpt-4.1-mini, sft-gpt-4.1-nano, sft-qwen-4b-instruct. Item counts range from 516 (GPT-4.1 Nano) to 1,425 (SFT GPT-4.1 Nano). Note this schema uses negation/status, whereas prompts/phenotype_prompt.txt asks for negated/severity/clinical_status/certainty; the prediction files are post-standardisation outputs of the proprietary pipeline, not raw LLM JSON.

7. data/hpo_timeline_prevalence.json — cohort-level HPO prevalence around GLP-1 initiation

```
meta:
  generated_at: "2025-12-31T16:06:09Z"
  total_patients: 15000
  windows_days: [30, 60, 90, 120, 180, 240, 300, 365]
  window_labels_before: ["prev_30", ..., "prev_365"]
  window_labels_after:  ["post_30", ..., "post_365"]
  overall_event_counts: {label: int} # encounters in that window (post_30: 129,246; prev_30: 42,979)
overall_by_window:      {label: {"counts": {HP_id: int}, "percent": {HP_id: float}}} # unused
overall_cumulative_by_window: {label: {"counts": {...}, "percent": {...}}} # used
```

percent = round(counts / total_patients * 100, 4) holds for every entry. Cumulative windows contain 2,623 (prev_30) to 5,010 (post_365) HPO codes. This is the only patient-level-derived GLP-1 cohort artefact in Part I; it is an aggregate (no patient identifiers).

8. clustering/cluster_hierarchy.json — concept-discovery taxonomy (not read by code)

Seven event domains, each {cluster_id: {"count": int, "label": str, "parent": str}}: adherence_barriers (48 clusters), adherence_recommendations (47), dose_change_event (48), hospitalizations (49), med_stop_event (48), med_switch_event (50), med_symptom_event (49). label is the LLM-generated fine cluster name, parent the LLM-generated coarse category. This is the serialised result of Fig. 2a’s two-level taxonomy (k-means over OpenAI embeddings, LLM labelling), but the embeddings, the clustering code and the note spans are not released.

9. adjudication/ — blind third-physician adjudication (not read by code)

- adjudication_decisions_blind.csv: visit_num, section, finding_index, finding, reviewerA, A_verdict, A_reason, reviewerB, B_verdict, B_reason, adjudication, timestamp — one row per disagreement on the 20-visit subset (sections assessment_status, medications, ...).
- recall_adjudications_blind.csv: visit_num, section, finding, flagged_by, term_in_note, adjudication, timestamp — reviewer-flagged misses with the adjudicator’s ok_to_omit / miss verdict.
- adjudication_notes.md: the adjudicating physician’s verbatim rationale.

These document the “adjudicated reference (20 visits)” column of Table 1 but are not joined to the MD JSON files by any script (the visit numbering differs: visit_num integers vs VISIT-NN).

10. prompts/ — fragments of the proprietary extraction prompts (not read by code)

- phenotype_prompt.txt: Python source excerpt of the phenotype agent: system message, HPO domain instructions, the full-visit prompt and the differential (delta) prompt returning added / modified / removed, plus _standardize which coerces LLM output into the common finding schema. References undefined constants (TEMPORAL_INSTRUCTIONS, NEGATION_INSTRUCTIONS, COMMON_FINDING_SCHEMA_BLOCK, ...).
- verification_prompt.txt: the LLM-judge prompt scoring each atomic statement 0–3 for support by the source text and 0–3 for ontology name-match quality.

11. Results — results/

Plain-text logs of each experiment’s stdout (the tables documented in CODE_REFERENCE.md) and four PNGs from experiment4. All are regenerated by code/run.sh.

Paper coverage and modularity — Part I (extraction validation)

What the published paper reports, what this repository (plus Part II) lets you reproduce, and what is not available anywhere.

1. Display-item map

Paper item	What it shows	Reproducible from	Status
Fig. 1	Cohort flow diagram (35.12M → 16,061 / 14,788)	Part II (step0a, step1 logs)	counts only; the figure script is not released
Table 1 (top)	GPT-5.2 Low vs physician reference: P/R/F1 per domain, adjudicated 20 visits and single reviewer 31-100	—	not reproducible : the per-visit physician scoring of the 100-visit set is not in data/; only the 3-reviewer subset (MD1-3) is
Table 1 (bottom)	PhenoTagger / BioNER / BERT Med NER vs ours: N, precision, negation error	experiment2.py Table 2, experiment3_med.py Table 2	reproducible (medication N comes from med_aggregate_stats.json)
Table 2	Verification-score distribution over 1M visits; fine-tuning cohort; analysis cohort	—	not reproducible; produced by the proprietary pipeline
Table 3	Baseline characteristics by glycemic stratum	Part II step0_analysis_population_table.py	code released, data not
Fig. 2a	UMAP + two-level taxonomy of medication-switch reasons	clustering/cluster_hierarchy.json (labels and counts)	taxonomy output released; embeddings, k-means, UMAP, LLM labelling code not released
Fig. 2b	Knowledge-graph node counts, structured vs +unstructured	—	not released
Fig. 2c	One patient’s atomic-extraction timeline	—	not released (patient-level)
Fig. 3a–e	Weight / HbA1c trajectories, KM curves	Part II conf_int/gap_120/step4_*, step8_*	code released, data not
Fig. 3f	PHQ-9, pain, waist trajectories	Part II add_unstructured/*	code released, data not
Extended Data Fig. 1–3	Platform schematics, delta-extraction example	—	illustrations; prompt fragments in prompts/ show the mechanism
Extended Data Fig. 4	LoRA training-loss curves	—	not released (Azure fine-tuning logs)
Extended Data Table 1	Inter-rater κ, three physicians, per model	experiment2.py Table 1, experiment3_med.py Table 1	reproducible (also adds PABAK and degeneracy flags the paper does not print)
Extended Data Table 2	GEE + Cox estimates, full vs structured-only	Part II	code released, data not
Extended Data Table 3	Model-predicted change at 3/6/9/12 months	Part II step4_predictive_plots*	code released, data not
Extended Data Table 4	Note-derived assessments	Part II add_unstructured/*	code released, data not
Extended Data Table 5	BiolarkGSC+ benchmark: base, SFT, PP, multi-agent	experiment3.py	reproducible for every LLM row; the “Encoders & early LLMs” rows are quoted from the literature
Supplementary Table 1	Attrition across persistence definitions	Part II step0a_samplesize_analysis.py	code released, data not
Supplementary Note 2, Listings 1–2	Extraction prompts	prompts/phenotype_prompt.txt	fragment only (delta extraction prompt is present; shared instruction constants are not)

In short: **the three validation tables that rest on frozen labels (Table 1 bottom, Extended Data Tables 1 and 5) reproduce exactly from this repository with no data access**. Everything that touches patient-level EHR data has released code (Part II) but no data, and everything that is the extraction platform itself has neither.

2. What the paper describes that has no code here

Component (paper section)	Available?	Notes
Agent-assisted longitudinal note extraction (Methods, “Agent-assisted...”)	No	Only prompts/phenotype_prompt.txt (one agent’s prompt builder)
Visit-to-visit delta tracking: added / modified / removed	Prompt only	_differential_prompt in the phenotype prompt file
Atomic-statement verification, 0–3 support score	Prompt only	prompts/verification_prompt.txt; no scoring code
Ontology mapping (UMLS, SNOMED CT, HPO, ICD, LOINC, RxNorm)	No	experiment3.py has an HPO loader usable for label/synonym/hierarchy lookup only
Concept discovery (embeddings, k-means sweep, silhouette, LLM labelling, UMAP)	Output only	clustering/cluster_hierarchy.json
Knowledge-graph construction and storage	No	The graph schema is described in prose only
Agentic analytic environment (VS Code agents, research skills)	No	Its product is the wide CSV Part II consumes
Fine-tuning (Azure LoRA for GPT-4.1 Mini/Nano; Qwen 2.5 4B on A40s)	No	Hyper-parameters are in the Methods; outputs of the tuned models are in data/sft-*.json

Component (paper section)	Available?	Notes
Distillation label generation (GPT-5 Mini over 100k visits)	No	
Multi-model aggregation	Yes	Union in <code>experiment3.py</code> lines 408-447
Task-based post-processing	Yes	<code>is_valid_phenotype</code> in <code>experiment3.py</code>
Physician review interface (Supplementary Note 1)	No	Its exports are <code>data/MD*.json</code>
Blind adjudication	Records only	<code>adjudication/</code>
Cohort construction, GEE, Cox, note-derived assessments	Part II	see the sibling repository

3. Modularity assessment

How much of this repository can be lifted out and reused as a unit?

Unit	Reusable as-is?	Coupling
<code>code/agreement.py</code>	Yes. Pure functions over <code>{item: {rater: label}}</code> with a 3-class encoding.	Only <code>N_CLASSES</code> and the Present/Absent/Rejected semantics are fixed.
HPO loader + ancestor closure (<code>experiment3.load_hpo</code> , <code>hpo_ancestors</code> , <code>resolve_hpo_id</code>)	Yes , with a copy-out.	Module-level globals populated at import; HP: prefix filter.
Three-layer matcher (<code>gold_expansion</code> , <code>check_match</code> , <code>is_informative</code>)	Yes , for any ontology that supplies labels, synonyms and <code>is_a</code> .	Depends on the module globals above and on <code>GENERIC_TOKENS / FUNCTION_WORDS</code> .
Macro/micro scorer (<code>update_name_class_stats</code> , <code>print_macro_row</code>)	Mostly.	Class = normalised surface string; lives inside <code>main()</code> as a closure.
Post-processing filter (<code>is_valid_phenotype</code>)	Domain-specific.	Encodes GSC+ conventions (no diseases, no genotypes, no “normal”).
Review-metric functions (<code>experiment2.calculate_*</code>)	Yes, given records with <code>is_missing / is_correct / original_negation / corrected_negation</code> .	Source names and file names are literals.
<code>experiment4.py</code>	Partly.	Window-label scheme <code>prev_N/post_N</code> , meta keys and the diabetes exclusion are literals; the <code>delta/slope/plot</code> logic is generic.
<code>run.sh</code>	Yes.	Generic step runner.

Overall: **moderately modular at the function level, not at all at the package level.** There is no `__init__.py`, no configuration file, no CLI for experiments 2 and 3, and every script resolves `../data` relative to its own location. Reuse means copying functions or editing literals, not importing a library. The one deliberate abstraction (`agreement.py`) exists because code review found two inconsistent estimators.

4. Honest limits of what the released numbers prove

- The physician-review data cover 24–27 visits per reviewer for phenotypes and a smaller medication set; the 100-visit single-reviewer validation in Table 1 is not released.
- `med_aggregate_stats.json` is an opaque count file; the per-item reviews behind `N=344/179/221` are not released, so medication precision cannot be re-derived or re-sliced.
- GSC+ predictions were generated upstream; the prompts and decoding settings that produced `data/base-*.json` are not fully recoverable from `prompts/`.
- Agreement κ for the GPT medication pipeline is degenerate (constant marginals); the paper reports a prevalence-adjusted value, which `agreement.py` prints as PABAK.

Reuse guide — running this validation on a different clinical question or on MIMIC

This repository validates *outputs* of an extraction pipeline. It does not contain the pipeline. So “reusing it” means: you build (or already have) an extractor, you collect physician labels and a gold corpus in the formats below, and you re-point four scripts at them. The outcome-analysis side of the paper is covered in the sibling repository `glp1_nature2026_a1c_analysis/docs/REUSE_GUIDE.md`.

1. What you get for free, and what you must write

Need	Status in this repo	What you have to write
Inter-rater agreement (κ, PABAK, degeneracy)	<code>code/agreement.py</code> , reusable unchanged	A loader that turns your review export into <code>{item_key: {reviewer: 0/1/2}}</code>
Precision / recall / negation-error metrics from physician review	<code>experiment2.py</code> functions, reusable	A review tool (or spreadsheet) that produces records with <code>is_correct</code> , <code>is_missing</code> , <code>original_negation</code> , <code>corrected_negation</code>
Benchmark against a public gold corpus with ontology-aware matching	<code>experiment3.py</code> matcher, HPO loader, macro/micro scorer	A gold parser for your corpus format; an OBO loader for your ontology if not HPO; a new validity filter
Multi-model union	<code>experiment3.py</code> lines 408-447	nothing
Prevalence-delta figure around an index event	<code>experiment4.py</code>	A script that produces <code>hpo_timeline_prevalence.json</code> from your patient-level extractions
The LLM extractor itself (prompts, delta tracking, verification, ontology mapping)	Absent	Everything: see §4
Concept discovery (embedding + clustering + labelling)	Output only	Everything
Knowledge graph	Absent	Everything

2. Adapting to a different clinical question (same data source)

The experiments are question-agnostic except for a handful of literals.

2.1 Physician-review experiments (`experiment2.py`, `experiment3_med.py`)

- Export reviews in the `data/MD*.json` schema (`DATA_SCHEMA.md` §1-2). The required fields are `visit_key`, `source`, a surface string, a within-visit position (`start_pos/end_pos` or `finding_index`), `reviewer`, `is_missing`, `is_correct`, `original_negation`, `corrected_negation`, `label`.
- Edit `SOURCE_DISPLAY` (and `MED_SECTIONS`) to name your extractors.
- Edit the file-name lists (`MD1.json ...`) or, better, replace them with a glob.
- If your task has no negation concept (for example lab values), set `original_negation == corrected_negation` everywhere; the “negation error” column will be zero and “other error” carries all incorrect extractions.
- Decide the agreement label semantics. The 3-class encoding (Present / Absent / Rejected) is what `agreement.py` assumes; for a binary correct/incorrect task use classes 0 and 2 only.

2.2 Gold-corpus benchmark (`experiment3.py`)

Change	Where
Different corpus format	rewrite <code>parse_gold_standard</code> to return <code>{doc_id: {"phenotypes": [(surface, concept_id)]}}</code>
Different ontology (SNOMED, RxNorm, ICD, MONDO...)	rewrite <code>load_hpo</code> to return <code>names</code> , <code>synonyms</code> , <code>parents</code> , <code>alt_ids</code> , <code>obsolete</code> dicts keyed by your IDs; drop the HP: prefix filter at line 60. The matcher only needs labels, synonyms and a parent map.
Different “what counts as an entity” protocol	rewrite <code>is_valid_phenotype</code> ; the current one implements GSC+ rules (no diseases, no genotypes, no normal findings)
Your models	edit the <code>model_files</code> list in <code>main()</code> and <code>combined_models</code>
Prediction schema	<code>prediction_tokens</code> reads <code>name</code> and <code>hpo_matched_name</code> ; map your fields onto those

Keep the three-layer matcher as is. It was tuned after review to reject clinically opposite pairs; the pinned test cases in `test_experiment3.py` are a good regression harness and should be extended with opposites from your domain (e.g. hyper/hypokalaemia, brady/tachycardia).

2.3 Prevalence deltas (`experiment4.py`)

Produce a JSON with the §7 schema of `DATA_SCHEMA.md`: for each window label `prev_N` / `post_N`, the per-concept count of patients with at least one extraction in the cumulative window, plus per-window encounter counts for normalisation. Change `EXCLUDE_CODES` to the concept(s) that are your indication rather than an outcome. Replace the HPO name map if you use another vocabulary.

3. Adapting to MIMIC (MIMIC-IV with MIMIC-IV-Note, or MIMIC-III)

MIMIC gives you the two inputs the paper’s platform starts from — structured tables and free-text notes — but not any of the intermediate products this repo evaluates. Expect to write the whole extraction layer.

3.1 Mapping MIMIC to the paper’s inputs

Paper concept	MIMIC-IV source	Notes
Patient	<code>hosp.patients (subject_id)</code>	<code>anchor_age</code> , <code>anchor_year</code> instead of DOB; ages > 89 are shifted

Paper concept	MIMIC-IV source	Notes
Visit / encounter	<code>hosp.admissions (hadm_id), icu.icustays, hosp.emar</code>	Ambulatory visits as in the paper do not exist; MIMIC is inpatient / ED
Clinical note	<code>mimic-iv-note discharge.csv, radiology.csv</code> (MIMIC-III: NOTEEVENTS with many note types)	One discharge summary per admission; no longitudinal outpatient narrative. Delta extraction across visits is only meaningful across <i>admissions</i>
Medications (structured)	<code>hosp.prescriptions, hosp.emar, hosp.pharmacy</code>	Drug names free text + NDC/GSN; map to RxNorm yourself (RxNav API or <code>rxnorm</code> tables)
Diagnoses	<code>hosp.diagnoses_icd</code> (ICD-9 and ICD-10)	billing codes, assigned at discharge
Labs (HbA1c etc.)	<code>hosp.labevents + hosp.d_labitems</code> (HbA1c itemid 50852; glucose 50931)	LOINC codes available in <code>d_labitems</code> for many items
Vitals / weight	<code>icu.chartevents</code> (admission weight itemid 226512, daily weight 224639), <code>hosp.omr</code> (outpatient weight, BMI, BP in MIMIC-IV ≥ 2.0)	<code>omr</code> is the only outpatient-style measurement table
Phenotypes / symptoms	none — must be extracted from notes	HPO is a reasonable target; the GSC+ benchmark transfers directly
Verification score	none	you implement the 0–3 judge (<code>prompts/verification_prompt.txt</code> gives the rubric)

3.2 Which experiments transfer directly

- **experiment3 (GSC+ benchmark):** runs unchanged; it does not touch MIMIC at all. Use it to pick and tune your extractor before touching PHI-bearing notes.
- **experiment2 / experiment3_med:** transfer once you have a review export. Sample N discharge summaries, run your extractor, have ≥ 2 clinicians label every extracted statement in the schema of §2.1, add missed items as `is_missing` rows, blind a third reviewer to adjudicate disagreements (the `adjudication/*.csv` files show the record layout the authors used).
- **experiment4:** transfers once you have per-patient extraction timelines; index event could be first insulin order, first ICU admission, etc. MIMIC time-shifting is per patient and consistent within patient, so “days from index” is valid.

3.3 What you must build (ordered)

1. **Note segmentation and de-duplication** — discharge summaries are long and sectioned; the paper’s extractors work on a visit note plus the prior visit for delta context.
2. **Domain extractors** (medication, phenotype, assessment at minimum). Use `prompts/phenotype_prompt.txt` as a template: it shows the output schema (`name, negated, severity, clinical_status, certainty, atomic_statement`) and the full-visit vs differential prompt pattern. The undefined constants (`TEMPORAL_INSTRUCTIONS, NEGATION_INSTRUCTIONS, DETAILS_INSTRUCTIONS, COMMON_FINDING_SCHEMA_BLOCK`) you write yourself.
3. **Atomic-statement verification** — second LLM pass scoring each statement 0–3 against the source span (`prompts/verification_prompt.txt`). Keep the score on every record; the paper reports the distribution (Table 2) and uses it as a filter.
4. **Ontology grounding** — HPO for phenotypes (reuse `experiment3.load_hpo` for label and synonym lookup, add an embedding or string-match step), RxNorm for drugs, LOINC for labs.
5. **Timeline assembly** — one record per (patient, day, concept) with provenance (note id, span). This is what both the prevalence JSON and the Part II wide CSV are derived from.
6. **Review tooling** — a UI or spreadsheet that shows note + statement + span and captures the verdict fields.
7. **Optional:** concept discovery (sentence embeddings $\rightarrow$ k-means with silhouette sweep $\rightarrow$ LLM labelling $\rightarrow$ UMAP), which only has its *output* here (`clustering/cluster_hierarchy.json`).

3.4 Pitfalls specific to MIMIC

- De-identification tokens (____) replace names, dates and some drug names; the authors’ notes show “REDACTED” medication names caused apparent misses (`adjudication/adjudication_notes.md`). Expect the same.
- No outpatient longitudinal narrative: “visit-to-visit delta” becomes “admission-to-admission”.
- Negation and family-history sections are dense in discharge summaries; the paper’s negation handling (polarity kept as attribute) is essential, and PhenoTagger-style tools fail here (83 % negation error in Table 1).
- Fine-tuning on MIMIC text requires the PhysioNet credentialed-data agreement and prohibits sending text to most hosted LLM APIs unless they are covered (Azure OpenAI under the PhysioNet responsible-use terms; check the current policy). Local models (Qwen 2.5 4B as in the paper) avoid the question.

4. Minimum viable reimplementation checklist

- ☐ Extractor producing `{doc_id: [ {name, hpo_matched_name, negation, status, atomic_statement} ]}` $\rightarrow$ run `experiment3.py` against GSC+.
- ☐ Verification judge producing a 0–3 score per statement.
- ☐ Review export in `MD*.json` schema $\rightarrow$ run `experiment2.py` (phenotypes) and a medication analogue.
- ☐ Per-window prevalence JSON $\rightarrow$ run `experiment4.py` with your indication in `EXCLUDE_CODES`.
- ☐ A wide per-patient-day CSV (see Part II the `DATA_SCHEMA.md` section) $\rightarrow$ run the outcome pipeline.

Part II — GLP-1 outcome analysis repository (glp1_nature2026_a1c_analysis)

Architecture — Part II (GLP-1 outcome analysis)

This repository is **Part II** of the code behind *Computable longitudinal patient journeys from structured and unstructured EHR data* (Kim, Foty et al., *Nature Medicine* 2026, doi:10.1038/s41591-026-04695-x). It is the statistical pipeline that turns one wide, per-patient-day CSV exported from the authors' knowledge graph into the weight, HbA1c, time-to-event and note-derived-assessment results of the paper. Part I (extraction validation) is the sibling repository `glp1_nature2026_extraction_validation`.

1. Where this repo sits in the paper's system

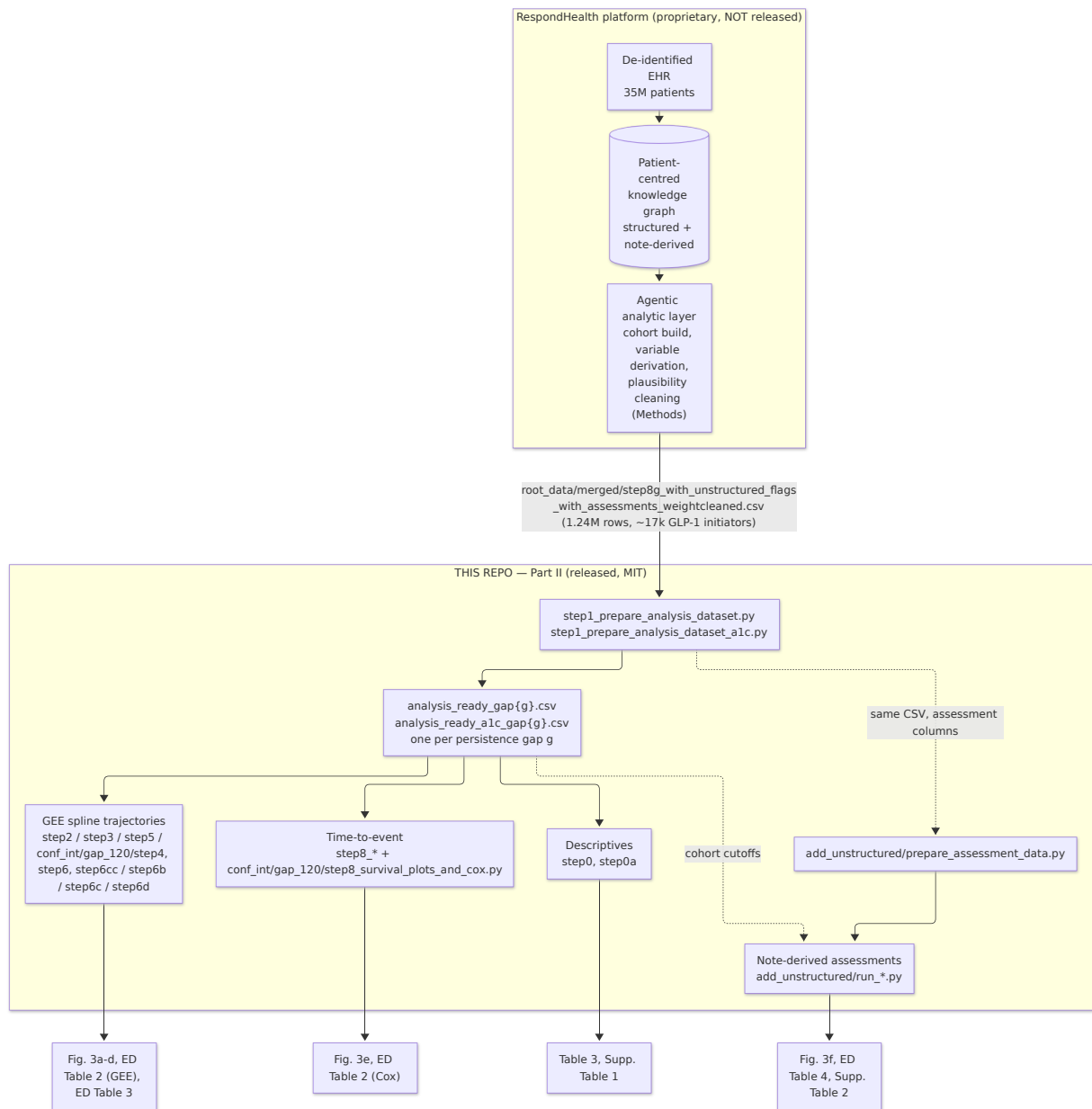


Diagram 1 — ARCHITECTURE.md

The repo contains no data. `root_data/` and `output/` are expected siblings of `code/` at a “study root” (`Path(__file__).resolve().parents[...]`), and every script is run from that root. The committed files live at the repository root, so the canonical layout is `<study_root>/code/` = this repo.

2. Pipeline topology

Three cohorts × nine persistence definitions × two outcomes, driven by one runner.

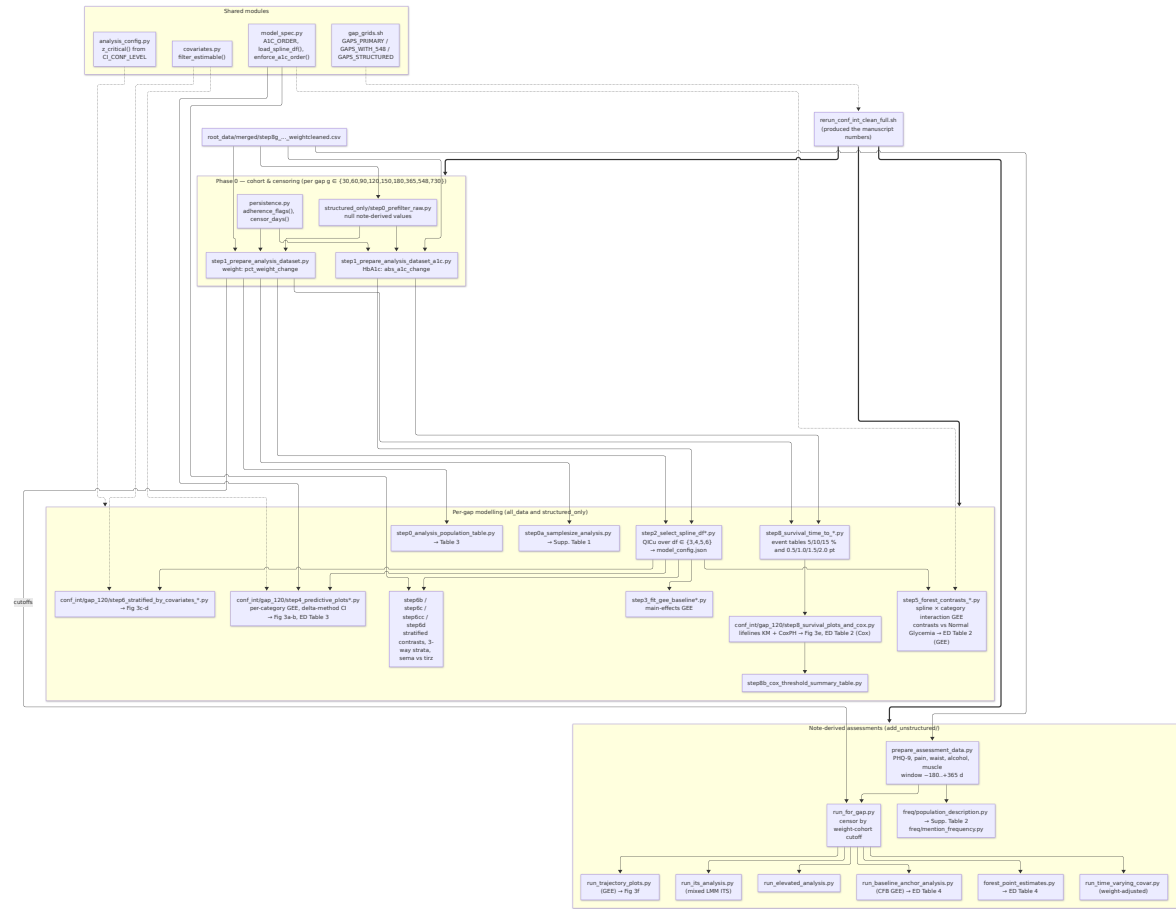


Diagram 2 — ARCHITECTURE.md

Runner phases (rerun_conf_int_clean_full.sh)

Phase	What runs	Cohort	Gap grid	Output root
0	step1 weight + A1c; structured_only/step0_prefilter_raw.py; step1 again on the prefiltered CSV	all-data and structured-only	GAPS_WITH_548 / GAPS_STRUCTURED	output/conf_int_clean/step1_*, structured_only_step1_*
1	prepare_assessment_data.py then the six add_unstructured/run_*.py scripts, twice (1_no_adherence, 1_no_adherence_full — identical)	note-derived, no persistence censoring	—	output/conf_int_clean/1_no_adherence*/
2	add_unstructured/run_for_gap.py --gap g	note-derived, censored by the weight cohort's cutoff	GAPS_WITH_548	output/conf_int_clean/gap_g/add_unstructured/
3	step4 pred (weight, A1c), step4 observed, step6 (weight, A1c), step6cc (weight, A1c), step8 TTE (weight, A1c), step8 KM+Cox	all-data	GAPS_WITH_548	output/conf_int_clean/gap_g/all_data/
4	the full step0 → step8b chain	structured-only	GAPS_STRUCTURED	output/conf_int_clean/gap_g/structured_only/
5	select_spline_df_assessment.py; optional PDF builders (not released); freq/*.py	—	—	output/conf_int_clean/spline_selection/

Phase 3 reads the spline configs from the *main* pipeline (output/gap_g/step2_select_spline_df*/), which must already exist; phase 4 regenerates them for the structured-only cohort. The other three runners (conf_int/run_all_gaps_all_data.sh, conf_int/run_all_gaps_structured_only.sh, add_unstructured/run_all_gaps.sh) re-run subsets of the same step scripts into output/conf_int/ or output/submitted_analysis/.

3. The one statistical pattern

Almost every modelling script is an instance of the same recipe, which is why the repo is large but conceptually small:

```
y ~ bs(days_from_baseline, df=best_df) [* baseline_a1c_category] + age_group + gender
+ baseline_bmi_final_category + race [+ weight_change_med]
GEE(family=Gaussian(), cov_struct=Independence(), groups=patient_id)
prediction grid: days 0..min(548, max observed) step 14, other covariates at their mode
CI:  $x \cdot \beta \pm z \cdot \sqrt{x' V x}$  with  $V = \text{robust cov\_params}()$ 
"centered" curves: subtract the day-0 prediction
```

Variants: per-category fits (step4, step6), interaction fits with contrasts (step5, step6c, step6d), stratified fits (step6b), 3-way strata (step6cc). Time-to-event uses the same baseline_a1c_category as the stratum, with events defined on the first crossing of a threshold and follow-up capped at 540 days. Note-derived assessments use the same GEE with a continuous age and C(...) covariates, plus mixed-effects interrupted-time-series models.

4. Design properties

- **Single-source rules.** Persistence (persistence.py), spline df (model_spec.load_spline_df, raises rather than defaulting), category order and reference level (model_spec.enforce_a1c_order), covariate estimability (covariates.filter_estimable), confidence level (analysis_config), gap grids (gap_grids.sh). All six were added in the v2 review response after duplicates were found to disagree.
- **Gap routing.** Scripts accept --adherence-gap-days, or parse gap[_]?(\d+) from the input path, and route output to output/gap_{g}/<step name>/ unless the outdir already contains /gap_.
- **Day-0 anchoring.** Weight trajectories get a synthetic day-0 row per patient (baseline_carried_to_day0 = 1, 18.5 % of patients at g = 120); HbA1c trajectories do not.
- **Fail-fast runner.** Completion markers (.step_complete) written only on exit 0; --resume re-runs half-finished steps; missing inputs are fatal.
- **No data, no end-to-end run.** tests/test_persistence_and_step0a.py runs on synthetic frames and is the only executable check without the licensed CSV.

See CODE_REFERENCE.md for every script, DATA_SCHEMA.md for the source CSV and all intermediates, PAPER_COVERAGE.md for the item-by-item map and modularity assessment, and REUSE_GUIDE.md for running this on another clinical question or MIMIC.

Code reference — Part II (GLP-1 outcome analysis)

Every script, what it reads, what it writes, and the exact model it fits. Line numbers refer to the committed files. Where several scripts share a pattern it is described once in ARCHITECTURE.md §3 and only the differences are listed here.

A. Shared modules

persistence.py — the persistence-of-therapy rule (327 lines)

Symbol	Line	Role
MENTION_VALUES = (1.0, 2.0), STOP_VALUE = 2.0	122, 125	Values of glp1_event_for_adherence that count as evidence / explicit stop
REQUIRED_COLUMNS	128	patient_id, glp1_event_for_adherence, glp1_days_from_baseline
normalize_patient_id(values)	133	Stripped strings; the only place IDs are canonicalised (fixes a dtype-mismatch defect from review)
mention_timeline(df)	159	{pid: [(value, day), ...]} sorted, non-negative days only
censor_day_for_patient(pairs, gap_days)	189	Per-patient last day within persistence: walk mention days, stop at the first inter-mention interval > gap (deliberately not > gap+1, lines 60-73), coverage ends prev + gap; capped at stop_day - 1
censor_days(source, gap_days)	248	Map over patients
adherence_flags(df, gap_days_list)	272	Adds adherence_{g} per row: 1 iff the most recent mention at or before the row is within g days and the row precedes any stop day

The rule, stated once (lines 19-33): an observation on day d is within persistence for gap g when some mention day $m \leq d$ has $d - m \leq g$, and $d < stop_day$. step1 uses adherence_flags (per row); step0a uses censor_days (per patient) because the attrition table counts calendar coverage, not retained observations. The two readings agree given the day-0 anchor that step1 guarantees. The study data never contain value 2, so the stop branch is unexercised.

model_spec.py (150 lines)

- A1C_ORDER = ["Normal Glycemia", "Prediabetes", "Type 2 Diabetes", "Poorly Controlled Diabetes"], REF_CATEGORY = A1C_ORDER[0].
- load_spline_df(config_json, key="best_df") — raises FileNotFoundError / KeyError / ValueError; **never defaults**, because the df is the model specification (ten scripts used to fall back to 3 silently).
- enforce_a1c_order(df, column, order, ref_category, require_ref, context) — ordered categorical; raises if the column is absent, no value matches, or the reference level is missing.

covariates.py (97 lines)

filter_estimable(candidates, df, *, exclude, context, logger) keeps a covariate iff it is not excluded, present, and has nunique(dropna=True) > 1; logs every drop with the reason. Replaces twelve inline list comprehensions that dropped silently.

analysis_config.py (87 lines)

CONF_LEVEL from env CI_CONF_LEVEL (default 0.95, validated in (0,1)); z_critical(level=None) = norm.ppf(0.5 + level/2); conf_level_label(). Every script sets Z_CRIT = z_critical() at import, replacing 61 hard-coded 1.96s.

gap_grids.sh

GAPS_PRIMARY=(30 60 90 120 150 180 365 730), GAPS_WITH_548 (adds 548), GAPS_STRUCTURED (= primary), GAP_PRIMARY=120, MAX_DAYS=730. Sourced by all four runners.

B. Cohort construction

step1_prepare_analysis_dataset.py — weight (905 lines)

CLI: --input-csv (default root_data/step8g_with_unstructured_flags.csv), --outdir (output/step1_prepare_analysis_dataset), --max-days (730), --adherence-gaps (nargs, default 90), --log-level.

load_and_prepare (line 335), in order:

- _fill_exclusion_flag for pregnant_during_glp1, bariatric_surgery_flag (absent → 0); drop rows with 1.
- Cohort filter: baseline_glp1_brand_final matches \bozempic\b|\bwegovy\b|\bmounjaro\b|\bzepbound\b **or** baseline_glp1_ingredient_final matches \bsemaglutide\b|\btirzepatide\b; fallback baseline_glp1 with either pattern (lines 39-50, 353-372). GLP1_BRAND_NAMES / GLP1_INGREDIENT_NAMES are the single source for these patterns.
- Require non-null baseline_weight_final **and** baseline_a1c_final (both pipelines need both baselines).
- event_date = date (else medication_date); compute pct_weight_change if absent; drop rows with missing outcome.
- _fill_covariates (Unknown / -1 fills), _derive_age_groups (decades; 20-39 vs 40+; 20-49 vs 50+).
- Ordered categorical baseline_a1c_category with Unknown, then drop Unknown.

7. `_ensure_glp1_evidence_and_metformin`: `glp1_evidence_therapy = is_glp1_event_med | is_glp1_event_clinical | event_type=="medication"`; `metformin_with_glp1_baseline` from medication_history JSON within ± 180 d of baseline, computed once per patient, **fails closed** on unparseable input (lines 155–232).
8. `days_from_baseline = (event_date - baseline_glp1_date).days`, keep $0 \leq d \leq \text{max_days}$.

`run_for_gaps` (line 702): `_ensure_baseline_rows` (synthetic day-0 row per patient lacking one, `pct_weight_change = 0.0`, `glp1_event_for_adherence = 1`, `baseline_carried_to_day0 = 1`, lines 430–528) → helper distance columns → `persistence.adherence_flags(df, gaps)` → for each gap, drop rows at or after the patient's first adherence_{g} == 0 day (`_rows_before_first_lapse`) → write `analysis_ready_gap{g}.csv` with the `keep_cols` list (lines 759–808). Raises if the marker column is missing from the output.

step1_prepare_analysis_dataset_alc.py — HbA1c (603 lines)

Imports the filters and helpers from the weight module. Differences:

- Requires `baseline_alc_final` and at least one of `alc_value` / `abs_alc_change`; derives the other (`abs_alc_change = alc_value - baseline_alc_final`).
- Missing-outcome rows are not dropped at load; instead patients are dropped unless they have ≥ 1 row with `days_from_baseline > 0` and a non-missing `alc_value` (lines 159–169).
- `_ensure_baseline_rows_alc` is the structural twin but creates no rows on the study data (HbA1c appears only on measurement dates; 38 % of patients have an observed day-0 value, the rest contribute no day-0 observation). `baseline_carried_to_day0` is emitted and sums to zero.
- Output `analysis_ready_alc_gap{g}.csv`; see `DATA_SCHEMA.md` §2 for the column differences.

structured_only/step0_prefilter_raw.py (110 lines)

Run on the raw CSV before step1 for the structured-only sensitivity cohort: where `glp1_has_structured == 0` set `glp1_event_for_adherence` to 0; where `weight_has_structured == 0` null `weight_in_pounds_final` and `pct_weight_change`; where `alc_has_structured == 0` null `alc_value`, `abs_alc_change`, `A1C_SOURCE`. Missing flag columns default to 0 (everything treated as unstructured). Reads the whole file into memory.

structured_only/gap120/step0_filter_to_structured.py (340 lines)

Alternative (post-step1) structured-only filter: keep rows with the flag set or `days_from_baseline == 0`, then patients with ≥ 1 post-baseline flagged row. Not used by the production runner.

C. Descriptives

step0_analysis_population_table.py (435 lines)

Table 3. Reads a weight analysis-ready CSV plus the matching A1c file (`usecols=["patient_id", "abs_alc_change"]` and `["patient_id", "baseline_alc_final"]`), optionally `root_data/step8f.csv` for brand/ingredient. One row per patient (first row after sorting by `baseline_glp1_date`). Rows: Mean/Median/Mode/SD(`ddof=1`)/Range of age, `baseline_weight_final`, `height_in_inches_final`, `baseline_bmi_final`, `baseline_alc_final`; counts of `baseline_bmi_final_category`, `age_group`, `gender`, `race`, `brand`, `ingredient`; achievement flags `min(pct_weight_change) ≤ -5/-10/-15` and `min(abs_alc_change) ≤ -0.5/-2.5`. Output `step0_baseline_table_gap{g}.csv` sorted alphabetically by variable, level (so clinical ordering is lost; the published table was re-ordered downstream). CLI: `--input-csv`, `--outdir`, `--adherence-gap-days (90)`, `--log-level`.

step0a_samplesize_analysis.py (336 lines)

Supplementary Table 1. For each `analysis_ready_gap*.csv` in `--analysis-dir`, takes the cohort (`patient_id`), rebuilds each patient's mention timeline from the **untrimmed source** (`--source-csv`, chunked 1.5 M rows; rows kept when `pct_weight_change` is non-missing and $0 \leq \text{date} - \text{baseline_glp1_date} \leq 730$), adds a synthetic (1, 0) mention for patients without an observed day-0 row, and counts patients with `persistence.censor_days(...)` $\geq 30 \cdot m$ for months 0–18. `--censor-from gapfiles` reproduces the submitted (double-censored) construction. Output `samplesize_by_month.csv` (`month_number`, `days_from_baseline`, `gap_30 ... gap_730`).

D. Trajectory models

step2_select_spline_df.py / _alc.py

Spline df selection. Formula `"{y} ~ bs(days_from_baseline, df={df}) + <covariates>"` for `df` in `--df-grid` (default 3,4,5,6), `covariates` = those of `age_group`, `gender`, `baseline_alc_category`, `baseline_bmi_final_category`, `race`, `weight_change_med` (A1c also `metformin_with_glp1_baseline`) with more than one level; GEE Gaussian / Independence / `groups=patient_id`; `res.qic()` → pick minimum **QICu**. Writes `df_selection*.csv` and `model_config*.json` (`best_df`, `best_qicu`). Production chose 3 for both outcomes. The A1c version has no gap routing.

step3_fit_gee_baseline.py / _alc.py

Main-effects GEE at `best_df` (read with plain `json.load`, not `load_spline_df`): `"{y} ~ bs(days_from_baseline, df) + age_group + gender + baseline_alc_category + baseline_bmi_final_category + race + weight_change_med + metformin_with_glp1_baseline"` (single-level covariates dropped with a log line; `--drop-covariate` to drop by name). Outputs `coefficients.csv` (`term`, `estimate`, `std_error`, `ci_lower`, `ci_upper`, `p_value`), `model_summary.txt`, `model_config_used.json`. The README's "`_nomet`" refers to running with metformin excluded; there is no flag or directory named `_nomet` — later scripts simply omit the term from their lists.

conf_int/gap_120/step4_predictive_plots.py (weight) / _alc.py — Fig. 3a-b, ED Table 3

Per baseline-A1c category (skip if `n_obs < --min-nobs 100`): `"pct_weight_change ~ bs(days_from_baseline, df) + age_group + gender + baseline_bmi_final_category + race [+ weight_change_med]"` (A1c: `abs_alc_change ~ ...`; `baseline_alc_category` is in the weight candidate list but is always dropped as single-valued inside a stratum). Prediction grid 0..min(548, max day) step 14 at modal covariates; delta-method CI. Writes per-category coefficient/covariance/overview CSVs, uncentered and centered trajectory PNGs, grouped overlays (Fig. 3a/b), and

tables/predicted_means_by_day_*.csv (ED Table 3 uses days 84, 183, 280, 365). A supported mask (≥ 40 obs and ≥ 30 patients within ± 28 d, ≥ 10 % of stratum, CI width ≤ 5 pp or $\leq 3 \times$ median) is written but not applied to the plots. Fixed y-limits (-10 , 4) weight, (-3 , 1) A1c. Env knobs `PLOT_CI_*`, `PLOT_SHOW_CI`, `PLOT_TRAJECTORY_CI_STYLE`.

`conf_int/gap_120/step4_observed_summary_plots.py`

Model-free companions: 90-day-bin and ± 30 -day-month means $\pm z \cdot sd/\sqrt{n}$ of `pct_weight_change`, `weight_in_pounds_final`, `alc_value`, `abs_alc_change` by category, anchored to the first bin. CLI `--weight-csv`, `--alc-csv`, `--outdir`, `--max-days` (548; runner passes 730), `--bin-width` (90).

`step5_forest_contrasts_weight.py` / `_alc.py` — ED Table 2 (GEE block)

One combined model with interaction: `"{y} ~ bs(days_from_baseline, df) * baseline_alc_category + age_group + gender + baseline_bmi_final_category + race [+ weight_change_med]"`. `enforce_alc_order` fixes the reference. Predictions at `--time-days` (weight 90, 180, 270, 365, 450, 548, 630, 730; A1c 90, 180, 365, 548) per category at modal covariates; contrast $(X_{cat} - X_{ref}) \cdot \beta$ with delta-method SE from the robust covariance. Outputs `gee_combined_coefficients.csv`, `forest_predictions.csv`, `forest_contrasts_vs_ref.csv`, forest PNGs per day and grouped 12 m / 18 m. Optional `step7_adherence_counts/adherence_counts.csv` (scope, day, `n_unique_window`) only annotates titles.

`conf_int/gap_120/step6_stratified_by_covariates_weight.py` / `_alc.py` — Fig. 3c-d

For each stratifier in `age_group`, `age_group_20_39_vs_40_plus`, `age_group_20_49_vs_50_plus`, `gender`, `race`, `metformin_with_glp1_baseline`, `baseline_bmi_final_category`, `glp1_user_group` (missing ones skipped) and each subgroup: `"{y} ~ bs(days_from_baseline, df) + <covariates minus the stratifier>"` with candidates `baseline_alc_category`, `baseline_bmi_final_category`, `age_group`, `gender`, `race`, `weight_change_med`. Grouped overlays per stratifier (Fig. 3c sex, 3d age). `stratified_summary_counts.csv`. A failing stratum is logged and skipped.

`step6b_stratified_contrasts_weight.py` / `_alc.py`

Per stratifier, separate GEE per subgroup; difference of predicted means at `--time-days` (default 365) vs a reference subgroup (youngest age bin, 20-39/20-49, else largest N); `se_diff = sqrt(se_ref^2 + se_sub^2)`. Output `contrasts_{strat_var}.csv`. `--min-nobs` 100, `--max-days` 548.

`step6c_stratified_forest_plots.py` (A1c) / `_weight.py`

Per stratifier $\times$ subgroup: (a) interaction model `bs(days) * baseline_alc_category + covariates` with contrasts vs Normal Glycemia, (b) main-effects model `bs(days) + baseline_alc_category + covariates` $\rightarrow$ `subgroup_predictions_main.csv`; per-day forests, grouped 12 m / 18 m forests (`day` $\in$ {365} and {548, 547, 540}), trajectories (weight: gender only). Known defects in the A1c version: `globals().get("alc_cats", [])` is always empty (line 1005), a `zip(y_ticks, texts_map.items())` label bug (line 1092), and the all-days PNG is overwritten by the grouped PNG. These affect supplementary figures only.

`conf_int/gap_120/step6cc_3waystrat_covariates_weight.py` / `_alc.py`

Sex $\times$ age (<40 / $40+$, from age or `age_group_20_39_vs_40_plus`) strata: `"{y} ~ bs(days_from_baseline, df=3) + baseline_alc_category + <remaining covariates>"`. Two defects: `getattr(args, "spline-df", 3)` never resolves the `--spline-df` argument, so `df` is always 3 (which equals the selected value); the A1c script contains a duplicated nested stratum loop (lines 367-536) that refits everything per outer iteration and leaves legend counts at 0. Figures only; no CSV.

`step6d_groups_by_glp1.py` (928 lines)

Semaglutide-only vs tirzepatide-only within each glycemic category: `"{y} ~ bs(days_from_baseline, df) * glp1_group2 * baseline_alc_category + covariates"`, rows truncated to `[0, --truncate-days 548]`. `glp1_user_group` is merged from `root_data/step8f.csv` when absent (it is absent from `step1` outputs); labels containing `sema/tirz` and not switch map to the two groups. Trajectories, differences and forests at days 90-548; all curves centred at the day-0 prediction (CIs shifted, not re-derived).

E. Time-to-event

`step8_survival_time_to_weight_loss.py` / `step8_survival_time_to_alc_drop.py`

Per patient and threshold (`WEIGHT_THRESHOLDS` = $[-5, -10, -15]$ %; `A1C_THRESHOLDS` = $[0.5, 1.0, 1.5, 2.0]$ pp): event = first day with `pct_weight_change` $\leq$ `thr` / `abs_alc_change` $\leq$ `-thr`, else censored at the last observed day; `MAX_FOLLOWUP_DAYS` = 540 converts any time beyond 540 to a censoring at 540. Writes the event table and a product-limit KM summary with Greenwood / $\log(-\log)$ median CIs. Required columns: `patient_id`, `days_from_baseline`, `outcome`, `baseline_alc_category`.

`conf_int/gap_120/step8_survival_plots_and_cox.py` — Fig. 3e, ED Table 2 (Cox block)

`KaplanMeierFitter` per category (x capped 548, lifelines' own 95 % band); `CoxPHFitter().fit(X, "time_days", "event")` with `pd.get_dummies(drop_first=True)` over `baseline_alc_category` (ordered, reference Normal Glycemia), `age_group`, `gender`, `baseline_bmi_final_category`, `race`, plus numeric `weight_change_med`; `proportional_hazard_test(..., time_transform="rank")`; HR CI $\exp(\text{coef} \pm z \cdot \text{se})$. Outputs `cox_*_by_baseline_alc.csv`, `models/cox_*_phreg.csv` (SAS-style for step8b), KM PNGs at 600 dpi. Quirk: covariates for **both** outcomes come from the A1c analysis CSV; weight-cohort patients absent from it get all-zero dummies (treated as reference).

`step8b_cox_threshold_summary_table.py`

Joins the step8 KM summaries (`n`, `n_events`) with the PHREG CSVs into `step8b_{weight,alc}_threshold_cox_summary.csv` (Outcome, Threshold, Baseline glycemic status, `e` / `n`, HR (LCI-UCI), `p`). Hard-codes output/gap_{g}/... paths, so the runners build a temporary symlinked output/ tree before calling it. `--out-markdown` names the output directory; no markdown is written.

F. Note-derived assessments (add_unstructured/)

Common conventions: no CLI; env AU_DATADIR (prepared data), AU_OUTROOT (output root), AU_STEP1 (weight analysis-ready CSV); BASE_COV_FORMULA = "age + C(gender) + C(race) + C(baseline_alc_category) + C(baseline_bmi_final_category)"; age median-imputed; GEE Gaussian / Independence; covid_era and (PHQ-9) antidepressant_baseline as extra 0/1 terms.

prepare_assessment_data.py (262 lines)

Reads the source CSV with usecols = ids, covariates and the six {d}_* columns per domain; window $-180 \leq \text{days} \leq 365$; a row is an observation of domain *d* iff {d}_present and {d}_value non-null; de-duplicate per patient-day (mean value); drop PHQ-2-only rows; derive post, time_months = days/30.44, time_post, has_both_periods, covid_era (baseline date in 2020-03-11..2022-05-11), and for PHQ-9 antidepressant_baseline from a second chunked pass over medication_history (26 keyword generics/brands, window $-180/+90$ d). Writes {d}_prepared.csv and domain_summary.json. No unit conversion or plausibility bounds live here; those are upstream.

run_for_gap.py (199 lines)

--gap g [--name add_unstructured] [--force]; env CONF_INT_DIR. Cutoff per patient = max days_from_baseline in output/step1_prepare_analysis_dataset/analysis_ready_gap{g}.csv (hard-coded main-pipeline path); keeps prepared rows with days < 0 or days ≤ cutoff for cohort patients; then runs the six analysis scripts as subprocesses with AU_* set.

run_trajectory_plots.py — Fig. 3f

Rows with has_both_periods == 1; "{val} ~ bs(days_from_baseline, df=3, include_intercept=False) + BASE_COV_FORMULA [+ antidepressant_baseline] [+ covid_era]"; grid $-180..365$ step 7; elevated overlays for PHQ-9 $\geq 5/\geq 10$, pain $\geq 4/\geq 7$ (pre-period mean), alcohol ≥ 12 . CI band suppressed where <3 patients within ± 21 d. Figures only.

run_baseline_anchor_analysis.py — ED Table 4 (change from baseline)

baseline_score = mean of values within ± 30 d; follow-up rows 30-365 d; change_score = value - baseline_score; per-window (1-3, 3-6, 6-9, 9-12 mo) one-sample t and Wilcoxon; LMM smf.mixedlm("change_score ~ time_months + covariates", groups=patient_id, reml=True); GEE with a synthetic day-0 anchor row (change_score = 0) and bs(days, df=3); grid 0..365 step 7; report baseline_anchored_report.md. The ED Table 4 point estimates themselves come from forest_point_estimates.py.

forest_point_estimates.py — ED Table 4

ITS-style estimate (full $-180..365$ GEE on has_both_periods rows) and CFB estimate (as above) at days 91/183/274/365: $\text{delta} = \hat{y}(t) - \hat{y}(0)$, $\text{se} = \sqrt{\text{se}_t^2 + \text{se}_0^2}$. Subgroups: PHQ-9 All/ $\geq 5/\geq 10$, pain All/ $\geq 4/\geq 7$, alcohol All/ ≥ 8.6 ("top 25 %"), waist and muscle All. covid_era is **not** included here, unlike the other scripts. Output point_estimates_3_6_9_12mo.csv.

run_its_analysis.py, run_elevated_analysis.py

Mixed-effects interrupted time series "{val} ~ time_months + post + time_post + covariates" (random intercept per patient, REML; PHQ-9 and pain only) plus paired pre/post tests for all five domains; the elevated variant restricts to pre-period mean $\geq$ threshold and drops the demographic fixed effects. Supplementary results. The elevated script's comparison panel looks for a stale path and labels and is never produced.

run_time_varying_covar.py

Adds concurrent pct_weight_change (nearest weight observation via merge_asof, not the ± 60 d the docstring claims) as a time-varying covariate; base vs weight-adjusted GEE and CFB; alcohol "top 25 %" is the run-time 75th percentile here. Outputs coefficient and point-estimate tables and an xlsx.

select_spline_df_assessment.py

QICu sweep over df 2-6 with no covariates per domain and gap; informational — the analysis scripts hard-code df = 3.

freq/population_description.py — Supp. Table 2; freq/mention_frequency.py

Module-level scripts (run on import). Population description of the PHQ-9 / pain / waist populations against the gap-120 weight and A1c cohorts; mention/encounter frequency counts per persistence definition. The production runner's exec_module path override is ineffective (module top level reassigns DATADIR/OUTDIR), so they read and write output/submitted_analysis/.

G. Runners

Runner	Grid	Notes
rerun_conf_int_clean_full.sh	GAPS_WITH_548 (all-data, note-derived), GAPS_STRUCTURED	Produced the manuscript numbers. --resume, --keep-going; set -euo pipefail; completion markers; CI_CONF_LEVEL inherited from the environment
conf_int/run_all_gaps_all_data.sh	GAPS_PRIMARY	Phase-3 subset into output/conf_int/gap_g/all_data/; does not pre-check configs
conf_int/run_all_gaps_structured_only.sh	GAPS_STRUCTURED	Phase-4 chain into output/conf_int/gap_g/structured_only/; line 391 references an unset \$FORCE under set -u, which aborts at step8b
add_unstructured/run_all_gaps.sh	GAPS_WITH_548	Does not set CONF_INT_DIR, so writes to output/submitted_analysis/

H. Tests

`python3 tests/test_persistence_and_step0a.py` (needs pandas, numpy, scipy; no data). Pins: integer patient IDs survive normalisation; the removed rule variants are absent; randomised equivalence of `sensor_day_for_patient` and `adherence_flags`; stop-day strictness; the abutting `gap + 1` boundary; the day-0 marker; `z_critical`; fail-closed metformin helper; exclusion-flag default. All pass on the committed code.

Data schema — Part II (GLP-1 outcome analysis)

The pipeline consumes one wide CSV and produces per-gap analysis-ready CSVs, model configs, coefficient tables, event tables and figures. None of the data is distributed; the schemas below are reconstructed from how the code reads and writes each file.

1. Source CSV — `root_data/merged/step8g_with_unstructured_flags_with_assessments_weightcleaned.csv`

One row per **patient** × **event-day** (a measurement, a medication event, or a note-derived assessment), long format, ~1.24 M rows. Columns are grouped by role. “Req.” marks columns whose absence stops the pipeline; the rest are optional with a documented fallback.

1.1 Identity and time

Column	Type	Meaning	Req.
patient_id	string (UUID)	Patient key; normalised to stripped string by <code>persistence.normalize_patient_id</code>	yes
date	date	Event date of the row; becomes <code>event_date</code>	yes (or <code>medication_date</code>)
medication_date	date	Fallback event date when <code>date</code> is entirely null	
baseline_glp1_date	date	Index date = first qualifying injectable GLP-1 RA event (day 0)	yes
days_from_baseline	int	Present in source for assessment rows; recomputed by <code>step1</code> as <code>(event_date - baseline_glp1_date).days</code>	
event_type	string	Row type; "medication" counts as GLP-1 evidence	

1.2 Exposure and persistence

Column	Type	Meaning	Req.
baseline_glp1_brand_final	string	Baseline brand; cohort filter matches <code>ozempic wegovy mounjaro zepbound</code> as whole words	one of the three
baseline_glp1_ingredient_final	string	Baseline ingredient; matches <code>semaglutide tirzepatide</code>	one of the three
baseline_glp1	string	Legacy agent label; fallback filter	one of the three
glp1_event_for_adherence	{0,1,2}	1 = evidence of ongoing GLP-1 therapy on this row, 2 = explicit stop (never present in study data), 0 = none	yes
glp1_days_from_baseline	int	Day index of that GLP-1 evidence	yes
is_glp1_event_med, is_glp1_event_clinical	0/1	Structured prescription vs note-derived evidence; OR-ed into <code>glp1_evidence_therapy</code>	
medication_history	JSON string	List of <code>{"medication_name", "medication_date"}</code> ; used for <code>metformin_with_glp1_baseline</code> (± 180 d) and <code>antidepressant_baseline</code> ($-180/+90$ d)	
glp1_user_group	string	e.g. "sema-only", "tirz-only", "switch..."; used by <code>step6d</code> and the assessment scripts; not propagated by step1 (<code>step6d</code> merges it from <code>root_data/step8f.csv</code>)	
glp1_has_structured, glp1_has_unstructured	0/1	Provenance flags; the structured-only prefilter zeroes <code>glp1_event_for_adherence</code> where <code>glp1_has_structured == 0</code>	

1.3 Outcomes

Column	Type	Meaning	Req.
weight_in_pounds_final	float, lb	Weight on this row	weight pipeline
baseline_weight_final	float, lb	Baseline weight (closest in $-60/+14$ d window)	both pipelines (A1c cohort also requires it)
pct_weight_change	float, %	$100 \cdot (\text{weight} - \text{baseline}) / \text{baseline}$; recomputed if absent	
a1c_value	float, %	HbA1c on this row	A1c pipeline (or <code>abs_a1c_change</code>)
baseline_a1c_final	float, %	Baseline HbA1c (lab or EAG-derived, per Methods)	A1c pipeline
abs_a1c_change	float, pp	<code>a1c_value - baseline_a1c_final</code> ; recomputed if absent	

Column	Type	Meaning	Req.
A1c_SOURCE	string	Source of the A1c value; nulled by the structured prefilter	
weight_has_structured, weight_has_unstructured, a1c_has_structured, a1c_has_unstructured	0/1	Provenance flags used by structured_only/	

1.4 Covariates

Column	Type / levels	Meaning
baseline_a1c_category	Normal Glycemia (<5.7 %), Prediabetes (5.7-<6.5), Type 2 Diabetes (6.5-<9.0), Poorly Controlled Diabetes (≥9.0), Unknown (dropped)	Effect modifier in every model; reference = Normal Glycemia
baseline_bmi_final_category	Underweight, Normal, Overweight, Obese I, Obese II, Obese III, Unknown	Covariate / stratifier
age	float, years (−1 when missing after step1 fill)	Source of age_group (decades), age_group_20_39_vs_40_plus, age_group_20_49_vs_50_plus, and step6cc's <40/40+
gender	M, F, Unknown	Covariate / stratifier
race	string, Unknown filled	Covariate; Supp. Table 2 collapses to Caucasian vs other
weight_change_med	0/1	Concomitant weight-affecting medication (RxNorm set, per Methods); covariate in every reported model
baseline_height_final, height_in_inches_final, baseline_bmi_final, BMI_final	float	Descriptives only

1.5 Exclusion flags

Column	Meaning
pregnant_during_glp1	1 → row excluded (NaN → 0; absent → 0)
bariatric_surgery_flag	1 → row excluded

Oral-semaglutide exclusion and the age ≥ 18 restriction (Fig. 1) happen **upstream**; they are not in this code.

1.6 Note-derived assessment columns (six per domain)

For each $d \in \{\text{phq9, pain_score, waist_circumference, alcohol, muscle_strength}\}$:

Column	Meaning
{d}_present	0/1 — an extraction of this domain exists on this patient-day
{d}_value	numeric value (PHQ-9 0-27; pain 0-10; waist inches; alcohol drinks/day; MRC 0-5), already unit-harmonised and plausibility-filtered upstream
{d}_min, {d}_max	min/max across extractions that day
{d}_n_rows	number of underlying NLP extractions
{d}_source_names	labels of the source instruments; PHQ rows matching phq-2 phq2 without phq-9 phq9 are dropped

2. Analysis-ready CSVs — analysis_ready_gap{g}.csv (weight), analysis_ready_a1c_gap{g}.csv

Long format, post-baseline rows only ($0 \leq \text{days_from_baseline} \leq 730$), censored at each patient's first non-persistent observation under gap g , one file per gap.

Column	Weight	A1c	Notes
patient_id, days_from_baseline	✓	✓	
baseline_carried_to_day0	✓ (1 for synthetic day-0 rows)	✓ (always 0)	
glp1_event_for_adherence, glp1_days_from_baseline	✓	✓	persistence inputs
days_since_last_glp1, days_since_last_glp1_evidence, days_to_{prev,next,nearest}_glp1, days_to_{prev,next,nearest}_glp1_evidence	✓	✓	helper distances, unused by models
adherence_{g}	✓	✓	1 for every retained row by construction
pct_weight_change	✓	—	outcome
baseline_a1c_final, a1c_value, abs_a1c_change	—	✓	outcome
baseline_a1c_category, baseline_bmi_final_category, age_group, age_group_20_39_vs_40_plus, age_group_20_49_vs_50_plus, gender, race, weight_change_med, metformin_with_glp1_baseline	✓	✓	covariates

Column	Weight	A1c	Notes
baseline_glp1, baseline_glp1_brand_final, baseline_glp1_ingredient_final, baseline_glp1_date, glp1_evidence_therapy	✓	✓	exposure
baseline_weight_final, weight_in_pounds_final	✓	✓	
age, baseline_height_final, height_in_inches_final, baseline_bmi_final, BMI_final	✓	—	descriptives
alc_has_*, weight_has_*, glp1_has_*	✓	✓	provenance

Not emitted: event_date, glp1_user_group, medication_history.

3. Intermediate and output artefacts

File	Producer	Schema
model_config.json / model_config_alc.json	step2	{"best_df": int, "best_qicu": float}; df_selection*.csv has df, QIC, QICu[, error]
coefficients.csv	step3	term, estimate, std_error, ci_lower, ci_upper, p_value
gee_combined_coefficients.csv	step5	term, Coef, CI Lower, CI Upper, p-value
forest_predictions.csv	step5, step6c	day, alc_group, pred, ci_low, ci_high, se, n_obs_window, n_unique_window
forest_contrasts_vs_ref.csv	step5, step6c	day, ref, alc_group, diff, ci_low, ci_high, se
tables/predicted_means_by_day_*.csv	step4 pred	alc_group, day, pred_mean_uncentered, ci_low_uncentered, ci_high_uncentered, pred_mean_centered, ci_low_centered, ci_high_centered, supported → ED Table 3 (days 84, 183, 280, 365)
summary_*_by_baseline_alc_{90day,months}.csv	step4 observed	binned observed means/SDs
contrasts_{strat_var}.csv	step6b	strat_var, ref, subgroup, day, diff, ci_low, ci_high, se, n_people_ref, n_obs_ref, n_people_sub, n_obs_sub
stratified_summary_counts.csv	step6	strat_var, subgroup, n_people, n_obs, scale
pooled_coefficients.csv, {cat}/{group}/predicted_trajectory.csv, forests/forest_day_{d}.csv	step6d	sema-only vs tirz-only
step8_weight_time_to_threshold_events.csv	step8 weight	patient_id, baseline_alc_category, threshold_pct ∈ {-5,-10,-15}, time_days (≤540), event
step8_alc_time_to_threshold_events.csv	step8 A1c	same with threshold_abs ∈ {0.5,1.0,1.5,2.0}
step8_*_time_to_threshold_summary.csv	step8	baseline_alc_category, threshold, n, n_events, median_time_days, median_time_days_lower_ci, median_time_days_upper_ci
cox_*_by_baseline_alc.csv	step8 KM+Cox	lifelines cph.summary + HR, HR_lower_95, HR_upper_95, ph_global_p_for_alc_block
models/cox_*_phreg.csv	step8 KM+Cox	SAS-PHREG-like: section, metric, value, Parameter, Level, HazardRatio, HRLowerCL, HRUpperCL, WaldChiSq, PrChiSq, covariate
step8b_{weight,alc}_threshold_cox_summary.csv	step8b	Outcome, Threshold, Baseline glycemc status, e / n, HR (LCI-UCI), p → ED Table 2 Cox block
step0_baseline_table_gap{g}.csv	step0	variable, level, Total, Normal Glycemia, Prediabetes, Type 2 Diabetes, Poorly Controlled Diabetes → Table 3
samplesize_by_month.csv	step0a	month_number, days_from_baseline, gap_30 ... gap_730 → Supp. Table 1
{d}_prepared.csv	prepare_assessment_data / run_for_gap	patient_id, days_from_baseline, {d}_value, {d}_min, {d}_max, {d}_n_rows, {d}_source_names, age, gender, race, baseline_alc_category, baseline_bmi_final_category, glp1_user_group, baseline_glp1_date, date, post, time_months, time_post, has_both_periods, covid_era [, antidepressant_baseline]
point_estimates_3_6_9_12mo.csv	forest_point_estimates	analysis ∈ {ITS, CFB}, subgroup, timepoint, day, n, estimate, se, ci_lo, ci_hi, domain → ED Table 4
tvc_coefs_{d}.csv, point_estimates_3_6_9_12mo_tvc.csv	run_time_varying_covar	weight-adjusted sensitivity
descriptive_summary.csv, model_coef_{d}.csv	run_its_analysis	paired pre/post and ITS coefficients
add_unstructured_population_description.{csv,xlsx}	freq/population_description	Supp. Table 2

4. Output directory layout (as produced by rerun_conf_int_clean_full.sh)

```
output/conf_int_clean/
├─ rerun_master.log, logs/
├─ step1_weight/analysis_ready_gap{30..730}.csv          step1_alc/analysis_ready_alc_gap{...}.csv
├─ prefiltered_structured_only.csv
├─ structured_only_step1_weight/...                      structured_only_step1_alc/...
├─ l_no_adherence/ (and l_no_adherence_full/, identical)
│   └─ data/{phq9,pain_score,waist_circumference,alcohol,muscle_strength}_prepared.csv, domain_summary.json
│   └─ ITS/{figures,tables,models}/, GEE/{figures/, CFB/{figures,tables}/, Forest/{figures,tables}/, time_var_covar/{figures,tables}/
├─ gap_{g}/
│   └─ add_unstructured/{data,ITS,GEE,CFB,Forest,time_var_covar}/
│   └─ all_data/{step4_predictive_plots, step4_predictive_plots_alc, step4_observed_summary_plots,
│   │           step6_stratified_by_covariates_{weight,alc}, step6cc_3way_by_age_sex_{weight,alc},
│   │           step8_survival_time_to_{weight_loss,alc_drop}, step8_survival_plots_and_cox}/
│   └─ structured_only/{step0_..., step2_..., step3_..., step4_..., step5_..., step6_..., step6b_..., step6c_..., step6cc_..., step6d_..., step8_..., step8b}/
└─ spline_selection/
```

Phase 3 additionally **reads** output/gap_{g}/step2_select_spline_df/model_config.json and ..._alc/model_config_alc.json from the main pipeline, and step6d / step0 fall back to root_data/step8f.csv (an earlier wide file with glp1_user_group and standardised brand columns) when those columns are missing.

Paper coverage and modularity — Part II (GLP-1 outcome analysis)

1. Methods-to-code map

Every analytic decision described in the Methods, with where it lives (or does not).

Methods statement	Implemented in	Status
Data source, de-identification, upstream normalisation to standard terminologies	—	upstream platform
Agent-assisted note extraction, delta tracking, verification, ontology mapping, concept discovery, KG	—	upstream platform (Part I validates its outputs)
Active adults, GLP-1 exposure via prescriptions + note mentions, 1 M-visit sample	—	upstream (Fig. 1 top boxes)
Age ≥ 18 , missing age/sex excluded; oral semaglutide excluded	—	upstream; not in this code
Pregnancy during window, bariatric history excluded	step1_* lines 343–351	here (flags consumed, not derived)
Injectable semaglutide / tirzepatide cohort (brand + ingredient)	step1_prepare_analysis_dataset.py 39–50, 353–372	here
Baseline window –60/+14 d, nearest value, ties on/before	—	upstream (baseline_*_final columns arrive computed)
Day-0 observation for every patient	_ensure_baseline_rows	here (weight only; see README §1)
Weight plausibility (45–200 % of median, 40 % spikes), BMI recalculation, HbA1c spikes, CDC outlier screen, EAG conversion	—	upstream; the input is already _weightcleaned.csv
HbA1c categories 5.7 / 6.5 / 9.0	—	upstream (baseline_a1c_category arrives labelled); order and reference enforced by model_spec
BMI categories	—	upstream
Concomitant weight-affecting medication covariate (RxNorm set)	—	upstream (weight_change_med arrives as a flag)
Metformin near baseline	_has_metformin_near_baseline	here (candidate covariate; excluded from reported models)
Persistence: backward gap rule, eight thresholds, g = 120 primary	persistence.py, gap_grids.sh	here
Follow-up ≤ 18 months	730 / 548 / 540 caps (README §5)	here
GEE, Gaussian, independence working correlation, robust SE, cluster = patient, B-spline 3 df chosen by QICu	step2, step3, step4, step5, step6*	here
Single pooled model with category as covariate (ED Table 2 GEE differences)	step3_fit_gee_baseline* (main effects) and step5 (interaction contrasts)	here
Separate models per category / sex / age for Fig. 3a–d, ED Table 3	conf_int/gap_120/step4_predictive_plots*, step6_stratified_by_covariates_*	here
KM curves, Cox PH with PH test; thresholds 5/10/15 % and 0.5/1.0/1.5 pt	step8_*, conf_int/gap_120/step8_survival_plots_and_cox.py	here (2.0 pt also computed; omitted from the paper as non-estimable)
Structured-only sensitivity	structured_only/step0_prefilter_raw.py + full chain in runner phase 4	here
Note-derived assessments: ± 30 d baseline, 30–365 d follow-up, GEE 3 df, LMM ITS, COVID indicator, antidepressant adjustment, time-varying weight	add_unstructured/*	here
Unit harmonisation and plausibility bounds for assessments (pain ≤ 10 , PHQ-9 ≤ 27 , MRC ≤ 5 , 1st–99th percentile)	—	upstream; not in this code
Elevated-baseline subgroups (PHQ-9 $\geq 5/\geq 10$, pain $\geq 4/\geq 7$, alcohol top quartile)	forest_point_estimates.py, run_trajectory_plots.py, run_time_varying_covar.py	here, with three different alcohol definitions (8.6, 12, run-time p75)
Two-sided tests, CIs, no multiplicity adjustment	analysis_config + each script	here

2. Display-item map

Item	Script(s)	Output file	Reproducib without data?
Fig. 1	counts from step1 logs, step0a	—	no (figure script not released)
Table 3	step0_analysis_population_table.py	step0_baseline_table_gap120.csv	no
Fig. 3a, b	conf_int/gap_120/step4_predictive_plots{,_alc}.py	grouped/grouped_trajectory_centered_outcome_{weight,alc}.png	no
Fig. 3c, d	conf_int/gap_120/step6_stratified_by_covariates_weight.py	by_gender/plots/grouped/..., by_age_group_20_39_vs_40_plus/...	no
Fig. 3e	conf_int/gap_120/step8_survival_plots_and_cox.py	km_*_by_baseline_a1c.png	no
Fig. 3f	add_unstructured/run_trajectory_plots.py (and CFB variants)	GEE/figures/trajectory_gee_{phq9,pain_score,waist_circumference}.png	no

Item	Script(s)	Output file	Reproducib without data?
ED Table 2 (GEE)	step5_forest_contrasts_{weight,a1c}.py	forest_contrasts_vs_ref.csv	no
ED Table 2 (Cox)	step8_survival_plots_and_cox.py → step8b	step8b_*_threshold_cox_summary.csv	no
ED Table 3	step4_predictive_plots*	tables/predicted_means_by_day_*.csv at days 84/183/280/365	no
ED Table 4	forest_point_estimates.py (+ run_time_varying_covar.py)	Forest/tables/point_estimates_3_6_9_12mo.csv	no
Supp. Table 1	step0a_samplesize_analysis.py	samplesize_by_month.csv	no
Supp. Table 2	freq/population_description.py	add_unstructured_population_description.csv	no
Supplementary figure sets (stratified forests, 3-way, sema vs tirz, ITS)	step6b/6c/6cc/6d, run_its_analysis.py, run_elevated_analysis.py	various	no

Not included at all: the xlsx/PDF table formatters, multi-panel figure assemblers, the patient-flow figure script (README scope note), and the build*_pdf.py builders the runner looks for in phase 5.

3. What is not available

1. **The data.** The single input CSV is licensed; nothing in the repo can be executed end to end without it. The regression tests are the only runnable artefact.
2. **The cleaning layer.** Everything the Methods describe under “Data preprocessing and plausibility criteria”, “Weight”, “BMI”, “HbA1c”, “Anthropometric cleaning and CDC outlier detection”, “Baseline definition” and “Derived variables” (categories, EAG conversion) is upstream. The CSV arrives cleaned and labelled.
3. **Cohort entry.** Age, oral semaglutide, “active adult”, and the 1 M-visit sampling are upstream.
4. **root_data/step8f.csv**, an earlier wide file that step0 and step6d fall back to for g1p1_user_group and brand columns.
5. **step7** (adherence counts; only annotates step5 titles).
6. **Deliverable formatting** scripts.

4. Modularity assessment

Layer	Modular?	Evidence
Persistence rule	Yes. One module, pure functions over three columns, tested, two documented readings.	persistence.py
Model specification guards	Yes. load_spline_df, enforce_a1c_order, filter_estimable, z_critical are small, importable, and used by most modelling scripts.	model_spec.py, covariates.py, analysis_config.py
Cohort construction	Partly. load_and_prepare is one function but mixes exclusion, cohort filter, outcome derivation, covariate fills, date handling and windowing; the A1c module reuses it by importing private helpers from the weight module. Agent names are a single constant.	step1_*
Trajectory modelling	No. The GEE recipe is re-implemented in ~14 scripts (_pred_ci, modal covariates, grid, centering, plotting), each with its own A1C_ORDER literal, y-limits, legend text and outdir routing. step3 keeps a private copy of the covariate filter.	step4, step5, step6*
Time-to-event	Partly. Event construction is isolated in step8_*; KM/Cox in one script; step8b depends on a hard-coded path tree.	step8*
Note-derived assessments	No. Seven standalone scripts with no CLI, env-variable configuration, duplicated covariate formulas, and three definitions of “top 25 % alcohol”.	add_unstructured/
Orchestration	Partly. One canonical runner with markers and fail-fast; three others overlap; phase 3 depends on a main-pipeline directory outside conf_int_clean/.	*.sh
Configuration	No central config. Column names, category labels, thresholds, horizons and paths are literals in each file; gap_grids.sh and CI_CONF_LEVEL are the only shared settings.	

Net: the review response (v2) made the *rules* modular (persistence, df, reference level, estimability, confidence) while leaving the *scripts* as a flat collection of near-duplicates. Reusing this for a new question is a find-and-replace exercise across ~40 files rather than a configuration change; see REUSE_GUIDE.md for the exact edit points.

5. Defects worth knowing before trusting a re-run

Verified against the committed code (none affects the primary reported estimates; see CHANGE_LIST.md/VERIFICATION.md references in the README for the authors’ own audit).

- conf_int/run_all_gaps_structured_only.sh:391 uses \$FORCE, never set, under set -u — the script aborts at step8b of the first gap. Use rerun_conf_int_clean_full.sh instead.

- `rerun_conf_int_clean_full.sh` phase 5 overrides `DATADIR/OUTDIR` on the `freq` modules before `exec_module`, which re-executes the module top level and resets them, so `mention_frequency.py` and `population_description.py` read/write `output/submitted_analysis/`, not `conf_int_clean/`.
- `step6cc_*`: `getattr(args, "spline-df", 3)` ignores `--spline-df` (`df` is always 3); the A1c script has a duplicated nested loop that refits every stratum once per outer stratum.
- `step8_survival_plots_and_cox.py`: Cox covariates for the weight outcome are taken from the A1c cohort file; weight-only patients get reference-level dummies. `_format_covariate_name_to_phreg` splits multi-underscore dummy names at the first underscore.
- `run_for_gap.py` and `freq/mention_frequency.py` always take cutoffs from `output/step1_prepare_analysis_dataset/` regardless of `CONF_INT_DIR`.
- `run_elevated_analysis.py`'s comparison panel reads a stale path and labels and is never produced.
- `run_time_varying_covar.py` matches the nearest weight in time, not within ± 60 d as its docstring says.
- Support masks in `step4_predictive_plots*` and `step6_*` are computed but not applied to the plots.

Reuse guide — same pipeline, different clinical question, or MIMIC

The pipeline is a generic “exposure-anchored longitudinal trajectory + time-to-threshold” design: index date, baseline window, persistence-based censoring, GEE splines by a baseline effect modifier, KM/Cox to thresholds, plus a note-derived-assessment module. Everything GLP-1-specific is in column names, label strings and constants. This guide lists, in order, what to produce, what to edit, and what you have to write from scratch.

1. The contract: produce one wide CSV

The entire pipeline starts from one long-format file (DATA_SCHEMA.md §1). Reproducing the paper on new data means producing that file. The minimum viable column set:

```
patient_id, date, baseline_glp1_date,
baseline_glp1_brand_final | baseline_glp1_ingredient_final | baseline_glp1, # cohort filter
glp1_event_for_adherence, glp1_days_from_baseline, # persistence
weight_in_pounds_final, baseline_weight_final, # outcome 1
alc_value, baseline_alc_final, # outcome 2
baseline_alc_category, baseline_bmi_final_category, age, gender, race, weight_change_med,
pregnant_during_glp1, bariatric_surgery_flag,
[*_has_structured, *_has_unstructured] # only for the sensitivity cohort
[{d}_present, {d}_value, {d}_min, {d}_max, {d}_n_rows, {d}_source_names] # only for note-derived assessments
[medication_history] # only for metformin / antidepressant flags
```

Everything the Methods describe as cleaning (plausibility ranges, spike removal, BMI recalculation, EAG → HbA1c, baseline-window selection, category assignment) must be done **before** this file exists. None of it is in the repo; §4 lists what to write.

2. Adapting to a different clinical question

Think of the design as five slots. Each slot maps to a small set of literals.

2.1 Exposure / index event

What	Where
Agent names used by the cohort filter	step1_prepare_analysis_dataset.py GLP1_BRAND_NAMES, GLP1_INGREDIENT_NAMES (lines 39–40); the A1c module imports the patterns
Index date column	baseline_glp1_date throughout (step1, step0, step0a, prepare_assessment_data.py)
Evidence-of-ongoing-therapy rows	glp1_event_for_adherence ∈ {0,1,2} and glp1_days_from_baseline; keep the names and the rule, just populate them from your exposure events (fills, orders, note mentions)
Persistence thresholds	gap_grids.sh; GAPS_DEFAULT in both step1 files
Agent sub-groups (step6d)	GLP1_TW0_GROUPS, the sema/tirz/switch substring mapping (lines 85, 466–476)

For a non-drug exposure (surgery, diagnosis, device) set `glp1_event_for_adherence = 1` on the index row only and use a single very large gap; the persistence machinery then reduces to “follow-up until last observation”.

2.2 Outcomes

What	Where
Continuous outcome 1 (pct_weight_change)	step1 weight: <code>_pct_weight_change</code> ; formula strings in step2/3/4/5/6*/6cc/6d weight scripts; y-limits (−10, 4) in step4/step6
Continuous outcome 2 (abs_alc_change)	step1 A1c lines 63–67; <code>_alc</code> scripts; y-limits (−3, 1)
Event thresholds	WEIGHT_THRESHOLDS, A1C_THRESHOLDS in step8_survival_time_to_*.py; file-name tags (5pct_loss, 0p5_reduction) in step8_survival_plots_and-cox.py and step8b
Sign convention	“improvement is negative” is assumed in step8 (<code><= thr</code>) and in step0’s achievement flags
Follow-up caps	730 (MAX_DAYS), 548 (–max-days, –truncate-days), 540 (MAX_FOLLOWUP_DAYS)

The simplest route is to keep the column names and write your outcome into `pct_weight_change` (percent change from baseline) and `abs_alc_change` (absolute change); then only labels and thresholds need editing.

2.3 Effect modifier (the “baseline glycaemic category”)

What	Where
Level names and reference	model_spec.A1C_ORDER and local copies in step0, step4 (with Unknown), step5, step6b A1c, step6, step8 Cox, population_description.py
Legend cut-points text	step4_predictive_plots* lines 246–252
Column name <code>baseline_alc_category</code>	~230 occurrences; keep the name, change the levels

2.4 Covariates

Candidate lists are literals in every modelling script: `age_group`, `gender`, `baseline_alc_category`, `baseline_bmi_final_category`, `race`, `weight_change_med` [, `metformin_with_glp1_baseline`]. `filter_estimable` will drop anything absent, so the quickest adaptation is to supply columns with those names (e.g. put your comorbidity flag in `weight_change_med`). Age grouping functions are in `step1_derive_age_groups`.

2.5 Note-derived assessments

DOMAINS in `prepare_assessment_data.py` (line 40) and the `DOMAINS` dicts in each `add_unstructured/run_*.py` (labels, scale ranges, elevated thresholds, spline df, which get an ITS model). The `{d}_present/_value/...` naming convention is the only coupling to the CSV. Change `ANTIDEP_KEYWORDS`, the COVID dates, and the `-180/+365` window as needed.

2.6 Practical order of work

1. Build the CSV (§1). Run `step1_*` for one gap and inspect `analysis_ready_gap*.csv`.
2. Run `step2` → `step3` → `conf_int/gap_120/step4_predictive_plots*` for that gap; this is Fig. 3a-b.
3. Run `step5` for ED Table 2, `step8` + `step8_survival_plots_and_cox.py` for Fig. 3e / ED Table 2.
4. Only then run the full runner across gaps.
5. Set `CI_CONF_LEVEL` once; never edit 1.96.

3. Adapting to MIMIC

MIMIC-IV (with `mimic-iv-note`) can feed this pipeline, but the paper's design is an **outpatient, multi-year, note-dense** cohort and MIMIC is an **inpatient / ED** database. Expect fewer repeat measurements per patient, shorter observable follow-up, and a different meaning of "persistence".

3.1 Table mapping for the wide CSV

CSV column	MIMIC-IV source	How
<code>patient_id</code>	<code>hosp.patients.subject_id</code>	cast to string
<code>date</code>	<code>hosp.labevents.charttime</code> , <code>icu.charthevents.charttime</code> , <code>hosp.omr.chartdate</code> , <code>hosp.prescriptions.starttime</code>	one row per measurement / order
<code>baseline_glp1_date</code>	first <code>hosp.prescriptions</code> (or <code>hosp.emar</code>) row whose drug matches your exposure; or <code>hosp.diagnoses_icd</code> for a diagnosis index	per patient
<code>baseline_glp1_brand_final</code> / <code>_ingredient_final</code>	<code>prescriptions.drug</code> , <code>gsn/ndc</code> → RxNorm	string-match or RxNav lookup
<code>glp1_event_for_adherence</code> , <code>glp1_days_from_baseline</code>	each <code>prescriptions/emar</code> row for the exposure → 1; a documented discontinuation → 2	<code>days = starttime - baseline</code>
<code>weight_in_pounds_final</code>	<code>icu.charthevents</code> itemids 226512 (admission weight, kg), 224639 (daily weight, kg); <code>hosp.omr</code> <code>result_name = 'Weight (Lbs)'</code>	kg × 2.20462; apply the Methods' plausibility rules yourself
<code>baseline_weight_final</code>	nearest weight in <code>-60/+14</code> d of index, ties on/before	per patient
<code>a1c_value</code>	<code>hosp.labevents</code> itemid 50852 (%)	
<code>baseline_a1c_final</code>	nearest in <code>-60/+14</code> d	
<code>baseline_a1c_category</code>	from <code>baseline_a1c_final</code> at 5.7 / 6.5 / 9.0	
<code>baseline_bmi_final_category</code>	<code>hosp.omr</code> BMI (kg/m ²) or weight/height ² (height itemid 226730, cm)	CDC classes
<code>age</code>	<code>anchor_age + (year(index) - anchor_year)</code> ; ages > 89 are 91	
<code>gender</code>	<code>patients.gender</code>	M/F
<code>race</code>	<code>admissions.race</code>	collapse to your scheme
<code>weight_change_med</code>	<code>prescriptions.drug</code> against your RxNorm list	per patient-window
<code>pregnant_during_glp1</code>	ICD-10 O00-O9A / ICD-9 630-679 in <code>diagnoses_icd</code> within the window	
<code>bariatric_surgery_flag</code>	ICD-10-PCS 0DB64Z3, 0D164ZA, ... or ICD-9-CM 43.89, 44.31, 44.38, 44.39 in <code>procedures_icd</code>	
<code>medication_history</code>	JSON of <code>prescriptions</code> rows per patient	needed only for the metformin / antidepressant flags
<code>*_has_structured</code> / <code>*_has_unstructured</code>	1/0 by source table (labs/chartevents vs note extraction)	
<code>{d}_present</code> , <code>{d}_value</code> , ...	from your own note extraction over <code>discharge.csv</code> / <code>radiology.csv</code> (MIMIC-III: <code>NOTEEVENTS</code>)	see §4
<code>glp1_user_group</code>	derived from the exposure drugs	only for step6d

Useful public helpers: the MIT-LCP `mimic-code` repository has concept SQL for weight, height, BMI and comorbidities; use it rather than re-deriving itemids.

3.2 Design adjustments you will need to make

- **Index event.** GLP-1 initiation in MIMIC is rare and inpatient; a more natural MIMIC question is e.g. "first insulin order", "first loop-diuretic order", "sepsis onset", with outcomes like creatinine, lactate, weight, or a note-derived score. The pipeline does not care.
- **Persistence.** Inpatient `emar` administrations are daily, so a 120-day gap is meaningless; use gaps of 1–3 days within an admission, or treat the admission as the follow-up window.
- **Time scale.** `days_from_baseline` can be hours ÷ 24 as floats; the splines and KM do not require integers. Replace the 730/548/540 caps and the 14-day grid step accordingly.
- **Day-0 anchoring** (`_ensure_baseline_rows`) assumes a baseline value exists within the window; with inpatient data most patients will have an observed day-0 row anyway.
- **Repeat measures.** GEE with independence working correlation and robust SE is fine for ICU density, but the spline df selection (QICu, grid 3–6) may prefer higher df; let `step2` choose.

- **De-identification.** MIMIC dates are shifted per patient into 2100–2200; `covid_era` is therefore meaningless and should be removed from the assessment formulas.
- **Race / sex.** `admissions.race` has ~30 levels; collapse before `step1` or `filter_estimable` will keep sparse levels and Cox dummies will explode.

3.3 The note-derived module on MIMIC

`prepare_assessment_data.py` expects the per-domain columns already extracted. For MIMIC you write the extractor (Part I's the `REUSE_GUIDE.md` section §3 covers it). Discharge summaries do carry pain scores, PHQ-type screens occasionally, weight/waist rarely, and alcohol use (drinks/day) in social history frequently; MRC muscle strength almost never. Choose domains accordingly.

4. What you must write yourself (not in either repository)

Ordered by how early you need it.

1. **Cohort and index-date builder** — identify exposure, first qualifying event, active-patient criteria, age/sex completeness, oral-formulation exclusion.
2. **Baseline-window selector** — nearest value in $-60/+14$ d with tie rule, for each baseline variable.
3. **Plausibility cleaning** — the full Methods list: height 48–90 in, weight 60–700 lb, BMI 10–80, 45–200 % of patient median, 40 % spike rule, 60 % of median baseline rule, BMI recalculation, HbA1c 3–20 % with the ± 365 -day spike rule, EAG $\rightarrow$ HbA1c $(EAG + 46.7)/28.7$, CDC percentile screen.
4. **Category assignment** — HbA1c and BMI classes, age groups (`step1` does age groups; the rest is upstream).
5. **Concomitant-medication flag** — RxNorm set for weight-affecting drugs $\rightarrow$ `weight_change_med`.
6. **Exposure evidence rows** — `glpi_event_for_adherence` from fills, orders and note mentions; explicit stops as 2.
7. **Provenance flags** — `*_has_structured` / `*_has_unstructured` per outcome per row.
8. **Note extraction + verification + ontology mapping** for anything note-derived (Part I guide), producing `{d}_present/_value/_min/_max/_n_rows/_source_names` and the `is_glpi_event_clinical` evidence rows.
9. **Assessment harmonisation** — units (waist to inches), bounds (pain ≤ 10 , PHQ-9 ≤ 27 , MRC ≤ 5), 1st–99th percentile trimming.
10. **root_data/step8f.csv equivalent** — only if you use `step6d` or `step0`'s brand fallback; otherwise add `glpi_user_group` to the main CSV.
11. **Deliverable formatting** — xlsx/PDF tables, multi-panel figure assembly, flow diagram.
12. **A configuration layer** (recommended) — the repo has none; a single YAML mapping of `{outcome columns, category levels, thresholds, caps, covariates}` would replace the per-script literals listed in §2.

5. Minimal smoke test without real data

Build a synthetic CSV with ~200 patients, 10–30 rows each, the §1 columns, and run:

```
python3 code/step1_prepare_analysis_dataset.py --input-csv synthetic.csv --outdir out/step1 --adherence-gaps 120
python3 code/step1_prepare_analysis_dataset_alc.py --input-csv synthetic.csv --outdir out/step1_alc --adherence-gaps 120
python3 code/step2_select_spline_df.py --input-csv out/step1/analysis_ready_gap120.csv --outdir out/step2 --adherence-gap-days 120
python3 code/step3_fit_gee_baseline.py --input-csv out/step1/analysis_ready_gap120.csv --config-json
output/gap_120/step2_select_spline_df/model_config.json --adherence-gap-days 120
python3 code/conf_int/gap_120/step4_predictive_plots.py --input-csv out/step1/analysis_ready_gap120.csv --config-json
output/gap_120/step2_select_spline_df/model_config.json --outdir output/gap_120/step4 --adherence-gap-days 120
python3 code/step8_survival_time_to_weight_loss.py --input-csv out/step1/analysis_ready_gap120.csv --adherence-gap-days 120
```

Note the gap routing: `step2` writes to `output/gap_120/step2_select_spline_df/` regardless of `--outdir` when `--adherence-gap-days` is given and the `outdir` lacks `/gap_`.